```
import numpy as np
import math, time, gc, pickle
from scipy.special import expi
from scipy.optimize import curve_fit
from scipy.signal import savgol_filter
from sklearn.linear_model import Ridge, RidgeCV
from functools import lru_cache

T_START = time.time()

# ============================================================
# 1. Segmented sieve
# ============================================================
def gen_primes(N, segment_size=8_000_000):
    if N <= 0: return np.array([], dtype=np.int64)
    upper_est = int(N * (math.log(N) + math.log(math.log(N)) + 2)) + 5000 if N >
    while True:
        limit_sqrt = int(math.isqrt(upper_est)) + 1
        base_sieve = np.ones(limit_sqrt + 1, dtype=bool); base_sieve[:2] = False
        for i in range(2, int(limit_sqrt**0.5) + 1):
            if base_sieve[i]: base_sieve[i*i::i] = False
        base_primes = np.flatnonzero(base_sieve)
        result = np.empty(N, dtype=np.int64)
        filled = 0; low = 2
        while low <= upper_est and filled < N:
            high = min(low + segment_size - 1, upper_est)
            seg = np.ones(high - low + 1, dtype=bool)
            for p in base_primes:
                if p * p > high: break
                start = max(p * p, ((low + p - 1) // p) * p)
                seg[start-low::p] = False
            if low <= 1: seg[:max(0, 2-low)] = False
            seg_primes = np.flatnonzero(seg) + low
            take = min(len(seg_primes), N - filled)
            result[filled:filled+take] = seg_primes[:take]
            filled += take; low = high + 1
            del seg, seg_primes
        if filled >= N: return result
        upper_est = int(upper_est * 1.5)



# ============================================================
# [COMPACT] Compressed prime storage -- 8x memory reduction
```

```
# ═══════════════════════════════════════════════════════════
# Problem: P_ALL as int64 takes 8 bytes per prime. At TRAIN_END=10^9 that
# is **8 GB** resident for the whole run -- the cause of out-of-memory
# crashes on Colab (12 GB total), before any computation starts.
#
# Solution: store **gaps**, not values. Every prime after 2 is odd, so
# gaps between them are always even => store gap/2 in a uint8 (one byte).
# Measured: max gap/2 over the first 20 million primes = 124, and the
# largest known gap below 2.3e10 is ~464 => gap/2 = 232 < 255. uint8
# suffices with margin.
#
# Retrieval: we keep anchor points every ANCHOR_STRIDE primes (negligible
# cost); any slice is reconstructed by cumsum from the nearest anchor.
# Verified: slices are **bit-identical** to the original array.
#
# Memory at TRAIN_END=10^9:  8.0 GB  ->  1.0 GB
ANCHOR_STRIDE = 1_000_000

class CompactPrimes:
    """Array-like interface over the primes, using 8x less memory.
    Supports len(), P[i], P[a:b], P[array] -- everything the model uses."""
    def __init__(self, P):
        # [FIX] The gap 2->3 equals 1, which is **odd** (2 being the only
        # even prime), so halving corrupted it. Fix: start compression at
        # p=3 and store 2 separately.
        self.n = len(P)
        self.odd0 = int(P[1]) if self.n > 1 else 3      # = 3
        Podd = P[1:]
        g = np.empty(len(Podd), dtype=np.uint8)
        g[0] = 0
        if len(Podd) > 1:
            d = np.diff(Podd) // 2
            if d.max() > 255:
                raise ValueError(f"gap too large for uint8: {int(d.max())*2}")
            g[1:] = d.astype(np.uint8)
        self.g = g
        self.anchors = Podd[::ANCHOR_STRIDE].astype(np.int64).copy()
    def __len__(self): return self.n
    def _range(self, lo, hi):
        """Returns P[lo:hi] exactly. Index 0 is the prime 2; the rest is
        reconstructed from gaps."""
        lo = max(0, lo); hi = min(self.n, hi)
        if hi <= lo: return np.empty(0, dtype=np.int64)
        head = np.array([2], dtype=np.int64) if lo == 0 else np.empty(0, dtype=np
        olo = max(lo-1, 0); ohi = hi-1          # indices within the odd part
        if ohi <= olo:
            return head
```

```
            ai = olo // ANCHOR_STRIDE
            base = int(self.anchors[ai]); s0 = ai*ANCHOR_STRIDE
            seg = self.g[s0+1:ohi]
            out = np.empty(ohi - s0, dtype=np.int64)
            out[0] = base
            if len(seg): out[1:] = base + 2*np.cumsum(seg.astype(np.int64))
            return np.concatenate([head, out[olo-s0:]]) if len(head) else out[olo-s0:
        def __getitem__(self, k):
            if isinstance(k, slice):
                st = 0 if k.start is None else (k.start if k.start >= 0 else self.n+k
                en = self.n if k.stop is None else (k.stop if k.stop >= 0 else self.n
                r = self._range(st, en)
                return r[::k.step] if k.step else r
            if isinstance(k, (int, np.integer)):
                k = int(k) if k >= 0 else self.n + int(k)
                return int(self._range(k, k+1)[0])
            idx = np.asarray(k)
            if idx.dtype == bool: idx = np.flatnonzero(idx)
            # group by anchor block to avoid reconstructing huge ranges
            out = np.empty(len(idx), dtype=np.int64)
            order = np.argsort(idx); si = idx[order]
            i = 0
            while i < len(si):
                ai = max(int(si[i])-1, 0) // ANCHOR_STRIDE
                j = i
                while j < len(si) and max(int(si[j])-1,0)//ANCHOR_STRIDE == ai: j +=
                lo, hi = int(si[i]), int(si[j-1])+1
                blk = self._range(lo, hi)
                out[order[i:j]] = blk[si[i:j]-lo]
                i = j
            return out
        def astype(self, dt): return self[:].astype(dt)

    def gen_primes_compact(N, segment_size=8_000_000):
        """
        [PEAK-FIX] Compresses **during** generation, so a full int64 array is
        never built.

        The first version generated everything then compressed -- so the peak
        stayed at 8N bytes (8 GB at N=10^9), which is what caused the crash;
        compression helped only **after** the peak.
        Now each sieve segment is converted to gaps immediately and discarded,
        so the peak equals the segment size only (~hundreds of MB) instead of
        the full array.

        Memory at N=10^9:  peak 8.0 GB  ->  ~1.1 GB
        """
```

```
if N <= 0: raise ValueError("N must be > 0")
upper_est = int(N * (math.log(N) + math.log(math.log(N)) + 2)) + 5000 if N >
while True:
    limit_sqrt = int(math.isqrt(upper_est)) + 1
    base_sieve = np.ones(limit_sqrt + 1, dtype=bool); base_sieve[:2] = False
    for i in range(2, int(limit_sqrt**0.5) + 1):
        if base_sieve[i]: base_sieve[i*i::i] = False
    base_primes = np.flatnonzero(base_sieve)

    # Layout (identical to CompactPrimes): g has length N-1, index j
    # corresponds to P[j+1]. g[0]=0 (the prime 3), and
    # g[j] = (P[j+1]-P[j])//2 for j>=1.
    g = np.empty(max(N-1, 1), dtype=np.uint8); g[0] = 0
    anchors = []
    n_odd = 0            # how many odd primes stored (P[1..])
    prev = None
    low = 2
    while low <= upper_est and (n_odd + 1) < N:
        high = min(low + segment_size - 1, upper_est)
        seg = np.ones(high - low + 1, dtype=bool)
        for p in base_primes:
            if p * p > high: break
            start = max(p * p, ((low + p - 1) // p) * p)
            seg[start-low::p] = False
        if low <= 1: seg[:max(0, 2-low)] = False
        sp = np.flatnonzero(seg) + low
        if prev is None and len(sp):
            sp = sp[1:]                       # skip the prime 2
        take = min(len(sp), (N-1) - n_odd)
        sp = sp[:take]
        if take:
            if prev is None:
                if take > 1:
                    d = np.diff(sp) // 2
                    if d.max() > 255: raise ValueError(f"gap > 510: {int(d.ma
                    g[1:take] = d.astype(np.uint8)
            else:
                d = np.empty(take, dtype=np.int64)
                d[0] = (sp[0] - prev) // 2
                if take > 1: d[1:] = np.diff(sp) // 2
                if d.max() > 255: raise ValueError(f"gap > 510: {int(d.max())
                g[n_odd:n_odd+take] = d.astype(np.uint8)
            k = ((n_odd + ANCHOR_STRIDE - 1)//ANCHOR_STRIDE)*ANCHOR_STRIDE if
            while k < n_odd + take:
                anchors.append(int(sp[k - n_odd])); k += ANCHOR_STRIDE
            prev = int(sp[-1]); n_odd += take
        low = high + 1
```

```
            del seg, sp
        if n_odd + 1 >= N:
            C = CompactPrimes.__new__(CompactPrimes)
            C.n = N; C.g = g[:N-1]; C.odd0 = 3
            C.anchors = np.array(anchors, dtype=np.int64)
            gc.collect()
            return C
        upper_est = int(upper_est * 1.5)


# ═══════════════════════════════════════════════════════════
# 2. li / inv_li
# ═══════════════════════════════════════════════════════════
_EI_LN2 = float(expi(math.log(2)))

def _NORM(a):
    """[NORM] Normalisation = ln^2(n), **kept as is**.
    Replacing it with sqrt(n)/ln(n) was tried (dispersion 1.08x vs 1.57x --
    more uniform *inside* the training range). But measurement on actual
    targets showed it **widens the interval 1.8x** (13,914 vs 7,846),
    because it grows faster under extrapolation beyond training.
    Lesson: lower dispersion **inside** training does not imply better
    extrapolation **outside** it. ln^2 is retained."""
    a = np.maximum(np.asarray(a, dtype=float), 3.0)
    return np.log(a)**2

def _NORM_S(v):
    v = max(float(v), 3.0)
    return math.log(v)**2

def li_val(x):
    if x <= 2: return 0.0
    return float(expi(math.log(x)) - _EI_LN2)

def li_vec(x_arr):
    x_arr = np.asarray(x_arr, dtype=float)
    out = np.zeros_like(x_arr)
    mask = x_arr > 2.0
    out[mask] = expi(np.log(x_arr[mask])) - _EI_LN2
    return out

def inv_li_vec(ns, iters=6):
    ns = np.asarray(ns, dtype=float)
    lns = np.log(np.maximum(ns, 2.))
    x = np.maximum(ns * (lns + np.log(np.maximum(lns, 1e-9))), 2.5)
    for _ in range(iters):
        lx = np.log(x)
```

```
        li_x = expi(lx) - _EI_LN2
        x = np.maximum(x - (li_x - ns) * lx, 2.5)
    return x


# ═══════════════════════════════════════════════════════════════
# [UPGRADE-R] Riemann R function instead of li
# ═══════════════════════════════════════════════════════════════
# R(x) = sum mu(k)/k * li(x^(1/k)) -- the most accurate known analytic
# approximation to pi(x).
# Why it beats li: li approximates J(x) = pi(x) + (1/2)pi(sqrt x) +
# (1/3)pi(cbrt x) + ..., i.e. it counts prime **powers** as well. Moebius
# inversion removes that excess.
# Cost: R needs ~7 expi calls instead of one. Solution: D(x)=R(x)-li(x) is
# a very smooth function -> tabulate it once on a fixed logarithmic grid
# and interpolate (error ~8e-6).
# The first Newton step is then **free**: if li(x0)=n exactly, then
# R(x0)-n = D(x0) directly.
# [R-FULL] All nonzero Moebius terms for 2 <= k <= 59 (36 terms). The list
# used to stop at k = 19 (12 terms), which is exact only for x <= 2**21
# (~2.1e6): beyond that the terms k = 21, 22, 23, 26, ... become active
# (x**(1/k) > 2) and were silently dropped. For x <= _D_XMAX = 1e18 every
# term with k >= 60 has x**(1/k) <= 2 and contributes exactly zero under
# li_2(x) = Ei(ln x) - Ei(ln 2), so this list is the complete R(x) on the
# supported range (beyond 1e18 the table below would clamp -- do not use).
# Measured against an exact evaluation of R^-1(n): the old list was off by
# 0.29 (n=1e6), 1.2 (n=1e8), 3.9 (n=1e12) in x; with the full list the
# error is the table's own (about 1e-4 for n <= 1e10). Table build: 0.35s
# -> 0.56s, once. The Newton steps below are unchanged on purpose: with
# the full list they already land within ~1e-4 of the exact inverse.
_MU_T = [(2,-1),(3,-1),(5,-1),(6,1),(7,-1),(10,1),(11,-1),(13,-1),
         (14,1),(15,1),(17,-1),(19,-1),(21,1),(22,1),(23,-1),(26,1),
         (29,-1),(30,-1),(31,-1),(33,1),(34,1),(35,1),(37,-1),(38,1),
         (39,1),(41,-1),(42,-1),(43,-1),(46,1),(47,-1),(51,1),(53,-1),
         (55,1),(57,1),(58,1),(59,-1)]
_D_LOGX = None; _D_VALS = None
_D_XMAX = 1e18; _D_NPTS = 200_000

def _D_exact(x):
    x = np.asarray(x, dtype=float); s = np.zeros_like(x)
    for k, mu in _MU_T:
        s += (mu/k) * li_vec(np.maximum(x, 2.0001)**(1.0/k))
    return s

def _ensure_D():
    global _D_LOGX, _D_VALS
    if _D_LOGX is None:
```

```
        lg = np.linspace(math.log(2.5), math.log(_D_XMAX), _D_NPTS)
        _D_LOGX = lg; _D_VALS = _D_exact(np.exp(lg))

def D_corr(x):
    x = np.asarray(x, dtype=float); _ensure_D()
    return np.interp(np.log(np.maximum(x, 2.5)), _D_LOGX, _D_VALS)

def R_vec(x):
    return li_vec(x) + D_corr(x)

def inv_R_vec(ns):
    ns = np.asarray(ns, dtype=float)
    x = inv_li_vec(ns)
    x = x - D_corr(x)*np.log(x)                    # free step
    lx = np.log(x)
    return x - ((li_vec(x) + D_corr(x)) - ns)*lx   # one expi call

def _li_asymptotic_val(x, K=10):
    """Asymptotic series approximation to li(x) -- cheap (no expi), but
    accurate only for large x."""
    lx = np.log(np.maximum(x, 2.))
    s = np.ones_like(x); term = np.ones_like(x)
    for k in range(1, K+1):
        term = term * k / lx
        s = s + term
    return (x/lx) * s


def inv_li_vec_fast_bulk(ns, small_thresh=100_000.0, cheap_iters=4, exact_iters=1
    """
    [SPEED-FIX-2] A faster substitute for inv_li_vec, intended specifically
    for the full C_EMP scan over millions of points. Idea: the same correct
    Newton step (same derivative 1/ln(x)), but using the cheap asymptotic
    li(x) series (no expi) for the early iterations, followed by a single
    correction with the true expi to remove the series bias.
    **Warning**: the asymptotic series breaks down for small x (verified:
    error reaches 89% at n=10) -- so any n below small_thresh is computed
    by the ordinary exact method unchanged (cheap anyway, being few points).
    Verified: max error ~1e-12 across a mixed range (small+large), far more
    accurate than required, and ~2.2x faster than inv_li_vec(iters=4) on
    large inputs.
    """
    ns = np.asarray(ns, dtype=np.float64)
    out = np.empty_like(ns)
    small_mask = ns < small_thresh
    if small_mask.any():
        out[small_mask] = inv_li_vec(ns[small_mask], iters=6)
```

```
    big_mask = ~small_mask
    if big_mask.any():
        nb = ns[big_mask]
        lns = np.log(np.maximum(nb, 2.))
        x = np.maximum(nb * (lns + np.log(np.maximum(lns, 1e-9))), 2.5)
        for _ in range(cheap_iters):
            lx = np.log(x)
            li_x = _li_asymptotic_val(x, K=10)
            x = np.maximum(x - (li_x - nb) * lx, 2.5)
        for _ in range(exact_iters):
            lx = np.log(x)
            li_x = expi(lx) - _EI_LN2
            x = np.maximum(x - (li_x - nb) * lx, 2.5)
        out[big_mask] = x
    return out


def fast_ridge_correction(x_arr, G60, ridge_coef):
    """
    [COMPLETE-FIX] An optimised substitute for
    build_*_features(...).dot(ridge_coef) -- computes the same result
    without materialising the full (N,120) matrix at all (direct
    accumulation into the result vector), in float32 internally, and
    sharing the common subexpression (sq/lx) across all 60 terms instead
    of recomputing it.
    Verified: max deviation ~3e-5 from float64 (negligible against actual
    residuals in the thousands), and ~7x faster than build_features + dot.
    This is what makes the full-range scan (every point up to TRAIN_END,
    not a sample) practical rather than taking many minutes.
    """
    x32 = np.asarray(x_arr, dtype=np.float32)
    lx = np.log(np.maximum(x32, np.float32(2.)))
    sq = np.sqrt(np.maximum(x32, np.float32(2.)))
    base = sq / lx
    G32 = G60.astype(np.float32)
    rc32 = ridge_coef.astype(np.float32)
    result = np.zeros(len(x32), dtype=np.float32)
    for k in range(len(G60)):
        g = G32[k]
        A = base / g
        th = g * lx
        result += rc32[2*k]*(A*np.cos(th)) + rc32[2*k+1]*(A*np.sin(th))
    return result.astype(np.float64)


# ============================================================================
# [UPGRADE-R] Analytic base: Riemann R function instead of li
```

```
# ═══════════════════════════════════════════════════════════════════
# R(x) = sum_{k>=1} mu(k)/k * li(x^(1/k)) -- the most accurate known
# analytic approximation to pi(x).
# Measured against the true primes: R^-1 is closer to truth than li^-1 by
# a factor of 1.4x to 62x depending on scale. Using it as the base shrinks
# the residuals, hence shrinks C_EMP, hence narrows the final interval.
#
# Practical problem: R(x) needs ~7 expi calls per point instead of one
# (7x slower over 500 million points = unacceptable). Solution:
# D(x) = R(x) - li(x) is a very smooth, slowly varying function, so we
# tabulate it once on a logarithmic grid and use np.interp.
# Verified: interpolation error 1e-4 (negligible against residuals in the
# thousands), and 14x faster.
# [R-FULL] identical to _MU_T above (all nonzero Moebius terms, k <= 59).
_MU_TERMS = [(2,-1),(3,-1),(5,-1),(6,1),(7,-1),(10,1),(11,-1),(13,-1),
             (14,1),(15,1),(17,-1),(19,-1),(21,1),(22,1),(23,-1),(26,1),
             (29,-1),(30,-1),(31,-1),(33,1),(34,1),(35,1),(37,-1),(38,1),
             (39,1),(41,-1),(42,-1),(43,-1),(46,1),(47,-1),(51,1),(53,-1),
             (55,1),(57,1),(58,1),(59,-1)]
_D_LOGX = None; _D_VALS = None


def inv_R_scalar(n):
    return float(inv_R_vec(np.array([float(n)]))[0])


# ═══════════════════════════════════════════════════════════════
# 3. Axler
# ═══════════════════════════════════════════════════════════════
def axler_bounds(n):
    """[FIX-1] The constant in the upper bound was 0 instead of 10.667.
    Reference: Christian Axler, "New Estimates for the nth Prime Number",
    arXiv:1706.03651 (J. Integer Sequences 22, 2019), Theorem 1.1.
    Measured impact at n=10,000,000: width 221,484 -> 16,186 (13.7x narrower).
    The lower bound already matched Theorem 1.2 (n>=2, unconditional) -- unchange
    Three tiers according to each theorem's validity range:
      n >= 46,254,381 : Theorem 1.1 (tightest proven)
      688,383 <= n    : Dusart (1.10)
      below           : Rosser-Schoenfeld 1962"""
    y = math.log(n); w = math.log(y)
    tr = (w - 2.) / y
    if n >= 46_254_381:
        cr_u = (w*w - 6.*w + 10.667) / (2.*y*y)
        upper = n * (y + w - 1. + tr - cr_u)
    elif n >= 688_383:
        upper = n * (y + w - 1. + tr)
    else:
```

```
        upper = n * (y + w - 0.5)
    cr_l = (w*w - 6.*w + 11.508) / (2.*y*y)
    lower = n * (y + w - 1. + tr - cr_l)
    return lower, upper

# ═══════════════════════════════════════════════════════════════
# 4. Correct zeta zeros
# ═══════════════════════════════════════════════════════════════
G60_PI = np.array([
    14.134725141734693790457251983562 5,
    21.022039638771554992628479593896 9,
    25.010857580145688763213790992562 8,
    30.424876125859513210311897530584 0,
    32.935061587739189690662368964074 7,
    37.586178158825671257217763480705 3,
    40.918719012147495187398126914633 4,
    43.327073280914999519496122165406 8,
    48.005150881167159727942472749427 7,
    49.773832477672302181916784678563 8,
    52.970321477714460644147296608880 8,
    56.446247697063394804367759476706 0,
    59.347044002602353079653648674992 2,
    60.831778524609809844259901824524 1,
    65.112544048081606660875054253183 6,
    67.079810529494173714478828896522 1,
    69.546401711173979252926857526554 7,
    72.067157674481907582522107969826 1,
    75.704690699083933168326916762030 5,
    77.144840068874805372682664856304 7,
    79.337375020249367922763592877116 1,
    82.910380854086030183164837494770 6,
    84.735492980517050105735311206827 6,
    87.425274613125229406531667850919 1,
    88.809111207634465423682348079509 5,
    92.491899270558484296259725241810 5,
    94.651344040519886966597925815208 0,
    95.870634228245309758741029219246 7,
    98.831194218193692233324420138622 4,
    101.31785100573139122878544794029 24,
    103.72553804047833941639840810869 52,
    105.44662305232609449367083241411 19,
    107.16861118427640751512335196308 60,
    111.02953554316967452465645030994 45,
    111.87465917699263708561207871677 07,
    114.32022091545271276589093727619 10,
    116.22668032085755438216080431206 47,
    118.79078286597621732297913970269 99,
```

```
    121.37012500242064591894553297049 98,
    122.94682929355258820081746033077 00,
    124.25681855434576718473200796613 01,
    127.51668387959649512427932376690 60,
    129.57870419995605098576803390618 00,
    131.08768853093265672356637246150 15,
    133.49773720299758645013049204264 07,
    134.75650975337387133132606415716 99,
    138.11604205453344320019155519028 24,
    139.73620895212138895045004652338 24,
    141.12370740402112376194035381847 53,
    143.11184580762063273940512386891 39,
    146.00098248676551854740250759642 46,
    147.42276534255960204952118501043 16,
    150.05352042078488035143246723695 94,
    150.92525761224146676185252467830 58,
    153.02469381119889619825654425518 54,
    156.11290929423786756975018931016 91,
    157.59759181759405988753050315849 88,
    158.84998817142049872417499477554 03,
    161.18896413759602751943734412936 95,
    163.03070968718198724331103900068 80
])
G60_PN = G60_PI.copy()

def build_pi_features(x_arr):
    x_arr = np.atleast_1d(x_arr); lx = np.log(np.maximum(x_arr, 2.)); sq = np.sqr
    X = np.zeros((len(x_arr), 2*len(G60_PI)), dtype=np.float32)
    for k, g in enumerate(G60_PI):
        A = sq/(lx*g); th = g*lx
        X[:, 2*k] = A*np.cos(th); X[:, 2*k+1] = A*np.sin(th)
    return X

def build_pn_features(li_arr):
    li_arr = np.atleast_1d(li_arr); lg = np.log(li_arr); sq = np.sqrt(li_arr)
    X = np.zeros((len(li_arr), 2*len(G60_PN)), dtype=np.float32)
    for k, g in enumerate(G60_PN):
        A = sq/(lg*g); th = g*lg
        X[:, 2*k] = A*np.cos(th); X[:, 2*k+1] = A*np.sin(th)
    return X

def bias_pi_scalar(x, A_BIAS_PI, B_BIAS_PI):
    return A_BIAS_PI*math.sqrt(x/math.log(max(x,3.))) + B_BIAS_PI

def dbias_pi_dx(x, A_BIAS_PI, B_BIAS_PI):
    h = max(x*1e-5, 1.0)
    return (bias_pi_scalar(x+h,A_BIAS_PI,B_BIAS_PI)-bias_pi_scalar(x-h,A_BIAS_PI,
```

```
def Fb_pi(x, A_BIAS_PI, B_BIAS_PI):
    # [UPGRADE-R] R is required here too, otherwise an inconsistency of size
    # D(x) arises against the calibration
    return li_val(x) + float(D_corr(np.array([float(x)]))[0]) + bias_pi_scalar(x,

def Fz_pi(x, A_BIAS_PI, B_BIAS_PI, ridge_coef):
    f = build_pi_features(np.array([x]))[0]
    return Fb_pi(x, A_BIAS_PI, B_BIAS_PI) + float(np.dot(f, ridge_coef))

# ═══════════════════════════════════════════════════════════════
# 5. lower_hull_tsn_base
# ═══════════════════════════════════════════════════════════════
try:
    from numba import njit
    _HAVE_NUMBA = True
except ImportError:
    _HAVE_NUMBA = False
    def njit(*a, **k):
        if len(a)==1 and callable(a[0]): return a[0]
        return lambda f: f

@njit(cache=True)
def _lower_hull_core(p_arr_int):
    """[FIX-4] The identical monotone-chain algorithm, but:
    - n is generated internally (not stored as a full float64 array)
    - P_ALL is passed as is (int64, no float64 copy)
    - the hull grows dynamically instead of pre-allocating N elements
    The old version allocated ~20 GB at TRAIN_END=500M (an actual crash).
    The result is **bit-identical** (zero difference), and ~67x faster."""
    N = p_arr_int.shape[0]
    cap = 4096 if N > 4096 else max(N, 1)
    hx = np.empty(cap, dtype=np.float64); hy = np.empty(cap, dtype=np.float64)
    top = -1
    for i in range(N):
        x = np.float64(i + 1); y = np.float64(p_arr_int[i])
        while top >= 1:
            x1 = hx[top-1]; y1 = hy[top-1]; x2 = hx[top]; y2 = hy[top]
            if (x2-x1)*(y-y1) - (y2-y1)*(x-x1) <= 0: top -= 1
            else: break
        top += 1
        if top >= hx.shape[0]:
            nc = hx.shape[0]*2
            nhx = np.empty(nc, dtype=np.float64); nhy = np.empty(nc, dtype=np.flo
            for j in range(hx.shape[0]): nhx[j] = hx[j]; nhy[j] = hy[j]
            hx = nhx; hy = nhy
        hx[top] = x; hy[top] = y
```

```
    return hx[:top+1], hy[:top+1]

def lower_hull_tsn_base(TRAIN_END, P_ALL, chunk=20_000_000):
    """
    [COMPACT-FIX] Previously: _lower_hull_core(P_ALL[:TRAIN_END]) -- which,
    with compressed storage, **rebuilds the entire int64 array** (8 GB at
    10^9), nullifying the compression entirely and crashing. It now works
    in chunks: each chunk is reconstructed, its hull built, then released,
    with the partial hull carried into the next chunk.
    The lower convex hull can be built incrementally because the monotone
    chain algorithm processes points in order and keeps only a small
    stack.
    """
    if not isinstance(P_ALL, CompactPrimes):
        hx, hy = _lower_hull_core(P_ALL[:TRAIN_END])
    else:
        hxs, hys = [], []
        for cs in range(0, TRAIN_END, chunk):
            ce = min(cs + chunk, TRAIN_END)
            blk = P_ALL[cs:ce]
            # prepend the previous partial hull to keep continuity
            if hxs:
                prev = np.array(hys[-1], dtype=np.int64)
                blk = np.concatenate([prev, blk])
                off = cs - len(prev)
            else:
                off = cs
            _hx, _hy = _lower_hull_core(blk)
            hxs = [_hx + off]; hys = [_hy]
            del blk, _hx, _hy
        hx, hy = hxs[0], hys[0]
    slopes = (hy[1:]-hy[:-1])/(hx[1:]-hx[:-1])
    fs = np.floor(slopes).astype(np.int64)
    hit = np.where(fs[1:]-fs[:-1] >= 2)[0]
    if len(hit): return float(slopes[hit[0]])
    return float(slopes[-1])

# ═══════════════════════════════════════════════════════════════
# 6. Calibration
# ═══════════════════════════════════════════════════════════════
C_EMP_CHUNK = 2_000_000     # [PORT-1] C_EMP scan chunk size (memory bound)
C_EMP_MIN_ARG = 10_000      # critical C_EMP cutoff (see [FIX-2]) -- single sourc
BOUNDARY_SEEDS = 5          # [PORT-10] number of seeds for the boundary search.
# The pi(x) cloud has gaps and the search logic steps over them, so the
# final bound used to depend on the seed

# ═══════════════════════════════════════════════════════════════
```

```
# [PRIMECOUNT] Alternative source for p(n) -- no sieving, no storage
# ================================================================
# Calibration needs p(n) at points inside the training range. The current
# approach sieves TRAIN_END numbers and stores them -- O(n) memory, which
# is the model's ceiling (~10^9 on 12 GB).
#
# primecount implements Meissel-Lehmer/Gourdon: it computes p(n) by
# combinatorial decomposition with O(x^(1/3)) memory and no storage.
# Actual measurement:
#     nth_prime(10^9) = 22,801,763,489  in 0.002s
#     100,000 calls   ~ 60 seconds
#     agreement with sieve: 100% (tested at 3 well-separated points)
#
# And C_EMP does not need every point -- measured on a sample:
#      30,000 points -> 95.4% of the full value
#     100,000 points -> 98.3%   (and at TRAIN_END=60M: 98.26% -- stable)
# The shortfall is covered by PC_MARGIN.
#
# Impact at TRAIN_END=10^9:
#     memory          : 1 GB    ->  ~0
#     calibration time: 3,712s  ->  ~60s   (60x faster)
#     ceiling         : 10^9    ->  primecount's own limit

# ================================================================
# [STREAM] Full C_EMP scan without storage -- full coverage, bounded memory
# ================================================================
# Problem: the only way to guarantee C_EMP is to inspect **every** point.
# A full sieve does that, but stores P_ALL (1-8 GB at 10^9), which is the
# model's ceiling. Sampling breaks the ceiling but loses the guarantee
# (a 2-5% shortfall covered by an estimated margin).
#
# Solution: sieve **in chunks** and compute C_EMP on each chunk as soon as
# it is generated, then discard it. Every point is genuinely inspected =>
# **full coverage, zero shortfall, no estimated margin**.
# Memory equals the chunk size only (~8 MB) regardless of TRAIN_END.
#
# Measured at TRAIN_END=20M: a full sieve pass takes 2.3s with an 8 MB
# peak per chunk (versus 2.9s + 160 MB for full storage).
#
# Two passes are needed: the first collects the calibration grid (to fit
# the coefficients), the second computes C_EMP -- because C_EMP requires
# ridge_coef and median to be already known.
def _sieve_stream(TRAIN_END, segment_size=8_000_000):
    """Generator yielding (start_index, prime_array) chunk by chunk,
    without storage."""
    up = int(TRAIN_END*(math.log(TRAIN_END)+math.log(math.log(TRAIN_END))+2))+500
         if TRAIN_END > 6 else 15
```

```
    while True:
        ls = int(math.isqrt(up))+1
        bs = np.ones(ls+1, dtype=bool); bs[:2] = False
        for i in range(2, int(ls**0.5)+1):
            if bs[i]: bs[i*i::i] = False
        bp = np.flatnonzero(bs)
        cnt = 0; low = 2; ok = False
        while low <= up and cnt < TRAIN_END:
            high = min(low+segment_size-1, up)
            seg = np.ones(high-low+1, dtype=bool)
            for p in bp:
                if p*p > high: break
                st = max(p*p, ((low+p-1)//p)*p)
                seg[st-low::p] = False
            if low <= 1: seg[:max(0, 2-low)] = False
            sp = np.flatnonzero(seg) + low
            take = min(len(sp), TRAIN_END-cnt)
            if take:
                yield cnt, sp[:take].astype(np.int64)
                cnt += take
            low = high+1
            del seg, sp
        if cnt >= TRAIN_END: ok = True
        if ok: return
        up = int(up*1.5)

def c_emp_streaming(TRAIN_END, ridge_coef, median_corr, side='pn',
                    A_BIAS=None, B_BIAS=None, segment_size=8_000_000):
    """C_EMP over **every** point, without storage. Full coverage, bounded
    memory."""
    best = 0.0
    for i0, sp in _sieve_stream(TRAIN_END, segment_size):
        nf = np.arange(i0+1, i0+1+len(sp), dtype=np.float64)
        if side == 'pn':
            bx = inv_R_vec_fast(nf)
            F = bx + fast_ridge_correction(bx, G60_PN, ridge_coef) + median_corr
            k = nf >= C_EMP_MIN_ARG
            if k.any():
                m = float(np.max(np.abs(sp[k].astype(np.float64) - F[k]) / _NORM(
                if m > best: best = m
            del bx, F, k
        else:
            xv = sp.astype(np.float64)
            F = (R_vec(xv) + (A_BIAS*np.sqrt(xv/np.log(np.maximum(xv,3.)))+B_BIAS
                 + fast_ridge_correction(xv, G60_PI, ridge_coef) + median_corr)
            k = xv >= C_EMP_MIN_ARG
            if k.any():
```

```
                m = float(np.max(np.abs(nf[k] - F[k]) / _NORM(xv[k])))
                if m > best: best = m
            del xv, F, k
        del nf, sp
    gc.collect()
    return best

def sieve_grid(TRAIN_END, grid_n, segment_size=8_000_000):
    """Collects a calibration grid (spread indices plus their values) in a
    single pass, without full storage."""
    idx = np.unique(np.linspace(0, TRAIN_END-1, grid_n).astype(np.int64))
    out = np.empty(len(idx), dtype=np.int64); w = 0
    for i0, sp in _sieve_stream(TRAIN_END, segment_size):
        hi = i0 + len(sp)
        j = w
        while j < len(idx) and idx[j] < hi: j += 1
        if j > w:
            out[w:j] = sp[idx[w:j] - i0]
            w = j
        del sp
        if w >= len(idx): break
    gc.collect()
    return idx, out



# ============================================================
# [SCALES] Reference calibration series -- embedded in the code
# ============================================================
# C_EMP values measured directly at 13 scales (doublings from 5K to
# 20.48M). Used to predict larger scales (mode F) when direct calibration
# is infeasible.
# Embedded here rather than in an external file: this is a fixed part of
# the method, and its absence made mode F fail completely at huge scales
# -- the very case it exists for.
# Add your own new direct calibrations here to improve prediction
# accuracy.
BUILTIN_SCALES = [5_000, 10_000, 20_000, 40_000, 80_000, 160_000, 320_000,
                  640_000, 1_280_000, 2_560_000, 5_120_000, 10_240_000, 20_480_00
                  500_000_000_000]
BUILTIN_C_EMP_PN = [0.0000, 0.7246, 1.2808, 1.8939, 2.6032, 3.3351, 3.8619,
                    4.2948, 7.3161, 10.1835, 11.2533, 14.5728, 18.9255,
                    1495.315]
BUILTIN_C_EMP_PI = [0.0464, 0.0679, 0.0693, 0.0993, 0.1215, 0.1491, 0.1820,
                    0.1969, 0.3005, 0.4053, 0.4422, 0.5354, 0.7463,
                    38.3170]
# The last point (5x10^11) was measured directly with mode C on Colab:
```

```
#   C_EMP_PN=1644.847 and C_EMP_PI=42.1487 were printed **after** applying
#   PC_MARGIN=1.10, so the raw values stored here are printed / 1.10.
#   Containment verified: p(10^12)=29,996,224,275,833 lies inside
#   [29,996,221,520,875 , 29,996,226,242,554] with a 41.7% edge margin,
#   and 116.65x narrower than Axler.
#   The exponent between it and 20.48M is 0.4325 -- consistent with the
#   0.377 seen within the series.


# ============================================================
# [PATTERN] Predicting C_EMP from a prior series of scales -- no full
# calibration
# ============================================================
# Problem: C_EMP requires p(n) at thousands of points. The per-call cost
# through primecount grows with n:
#     n=10^9 : 5.8ms   => 100K-point calibration = 10 minutes
#     n=10^12: 151ms   => 4 hours
#     n=10^13: 622ms   => 17 hours
# So the practical ceiling is ~10^12-10^13.
#
# Idea: calibrate on a series of **doublings** (5K, 10K, 20K, ...), infer
# the pattern, then predict the next scale instead of calibrating it.
#
# What measurement over 12 scales (10K to 20.48M) showed:
#   - C_EMP updates in **clusters**, not gradually: only 20 clusters over
#     20 million points (451 grouped updates), with millions of points
#     between them showing no change at all.
#   - coefficient of variation of the gaps = 1.46 (above pure Poisson 1.0)
#     => the clusters are **clustered**, not regular, so a jump is
#     possible at any time.
#   - but as the scale grows:
#       * clusters become rarer  (position-gap correlation r=+0.52)
#       * increments shrink      (position-increment correlation r=-0.45)
#     first 5 increments: 31.8% 2.5% 3.8% 37.1% 3.1%
#     last  5 increments:  1.7% 15.4% 0.2%  2.6% 3.5%
#
# Local prediction accuracy (from the last two scales, over 10
# predictions):
#     mean error +4.8%, worst undershoot -34.7%, worst overshoot +26.0%
#     and the last two predictions: -14.7% and -0.3%
#     => accuracy improves with scale
#
# Hence the margin **decreases with scale**: it covers the worst measured
# undershoot at small scales, and tightens at large ones where the
# prediction proved more accurate.
#
# **Warning:** this is an estimate, not a guarantee. C_EMP is a
```

```
# maximum-value statistic and the clusters are clustered, so a jump larger
# than any measured is possible at any scale.
GRID_TIME_BUDGET = 900.0    # [AUTO-GRID] time budget for building the grid (seco
GRID_MIN, GRID_MAX = 500, 20_000   # grid-size limits

# [MARGIN] The margin is derived statistically from the distribution of
# prediction error, not chosen by hand.
#
# **The model's actual range is a 2x jump only.** Training on [1, N]
# yields a valid containment on [N+1, 2N] and no further. So the
# distribution is measured at jumps <= 2x exclusively -- measuring it over
# longer jumps mixes in cases outside the operating range and spuriously
# inflates sigma (0.54 when mixing all jumps versus 0.147 when correctly
# restricted).
#
# Measured distribution of log(predicted / actual) at jumps <= 2x:
#     mu = +0.0172   sigma = 0.1473   (23 cases, scales 10K-20M)
#
# Margin = exp(z*sigma - mu):
#     95%   confidence -> 1.253
#     99%   confidence -> 1.385
#     99.7% confidence -> 1.529
#
# **Cross-region validation** (what distinguishes this from hand-tuning):
#   derived from 10K-20M and tested on an independent series 25M-62M that
#   never entered the derivation: **zero failures out of 9 cases** at all
#   three confidence levels.
#   The margin actually required there was 1.179, so 1.385 covered it with
#   17% to spare.
#
# The two former values (1.55 and 1.25) were hand-picked after seeing the
# results, with no derivation and no cross-region validation.
MARGIN_Z       = 3.00       # nominal confidence (3.0=99.7%, 2.33=99%, 1.65=95%)
# [CORRECTION] It was 2.33, giving a margin of 1.68. The only available
# paired measurement (at 2^40, where modes C and F were both run at the
# same scale) showed the prediction **undershot by 43%**: predicted
# 2,102.6 versus measured 3,681.8, so ~1.75 was required while the margin
# was 1.68 -- i.e. **4% short**.
# Containment still succeeded, but thanks to internal slack in the
# envelope (+/-2*C_EMP*ln^2), not to margin accuracy. Raising it to z=3.0
# makes it cover the available measurement.
#
# **Warning:** the confidence level is nominal, not real. The formula
# exp(z*sigma - mu) assumes a normal error distribution, which is
# **unverified** -- indeed C_EMP is a maximum-value statistic, and such
# distributions belong to a different, heavier-tailed family (Gumbel). The
# figure 99.7% therefore describes the formula, not reality.
```

```
# -- Paired measurements: prediction (F) versus direct measurement at the
#    **same scale** --
# Each row: (scale, measured C_EMP, raw predicted C_EMP)
# The measurement comes strictly from inside [1, N] -- by full sieve or by
# a primecount sample.
# The prediction uses only scales **smaller than N** (no data at N itself).
# The ratio (measured / predicted) is the margin actually required at that
# scale.
#
# This is the sole source of MARGIN_FLOOR: not a chosen number, but the
# largest ratio observed. Add any new paired measurement here and the
# margin updates automatically.
MARGIN_CALIBRATION = [
    (2_000_000,          8.009,     10.309),   # ratio 0.777 (prediction overshot
    (4_000_000,          9.696,     12.600),   # 0.770 (overshot)
    (8_000_000,         14.313,     12.001),   # 1.193
    (12_000_000,        15.237,     15.461),   # 0.986
    (20_000_000,        19.224,     18.705),   # 1.028
    (2**40,           3681.800,   2102.565),   # 1.751  <- worst observed
]

def _margin_floor_from_data(safety=1.00):
    """
    The largest margin observed to be actually required, from
    MARGIN_CALIBRATION.

    safety=1.00 means no manual inflation whatsoever -- the value is the
    worst measured, nothing more. (An earlier version used 1.85 = 1.751
    plus a 6% guess; that increment had no source.)

    The six current measurements span ratios 0.770 - 1.751, mean 1.084.
    In four of them the prediction already overshot (ratio < 1), so no
    margin was needed there.

    **Caveat:** five of the six lie at scales 2M-20M, and only one at
    2^40 -- which is the worst. The possibility that the required margin
    grows with scale remains open, and one point cannot settle it.
    """
    if not MARGIN_CALIBRATION: return 1.20
    return max(m/p for _, m, p in MARGIN_CALIBRATION if p > 0) * safety

MARGIN_FLOOR   = _margin_floor_from_data()
MARGIN_MAX_JMP = 2.5        # largest jump admitted (the model's operating range)
MARGIN_MIN_N   = 8          # minimum number of cases to accept a measurement

def measure_margin_params(scales, values):
    """
```

```
    Computes (mu, sigma, n) of the prediction-error distribution **from
    the given series** -- not from constants.

    Method: for every pair within the series whose jump is
    <= MARGIN_MAX_JMP, predict the second point from the first using the
    same algorithm as predict_c_emp, and record log(predicted / actual).
    mu and sigma are the statistics of that distribution.

    Why measured on every call rather than frozen? Because the series
    grows: each new direct calibration added changes the distribution.
    Freezing (mu, sigma) would make the rule operate on stale inputs and
    ignore every later measurement -- losing its purpose and becoming a
    fixed number dressed as a formula.

    Restricting to jumps <= 2.5x is deliberate: the model's operating
    range is [N+1, 2N], and longer jumps are not use cases. Including them
    spuriously inflates sigma (0.54 when mixed versus 0.147 when properly
    restricted).
    Returns None if there are too few cases, in which case the margin
    falls back to MARGIN_FLOOR.
    """
    s = np.asarray(scales, dtype=float); v = np.asarray(values, dtype=float)
    ok = v > 0; s, v = s[ok], v[ok]
    if len(s) < 3: return None
    rs = []
    for i in range(2, len(s)):
        if s[i-1]/s[i-2] > MARGIN_MAX_JMP: continue
        k = float(np.clip(math.log(v[i-1]/v[i-2]) / math.log(s[i-1]/s[i-2]), 0.0,
        for j in range(i, len(s)):
            if s[j]/s[i-1] > MARGIN_MAX_JMP: continue
            raw = v[i-1] * (s[j]/s[i-1])**k
            if raw > 0 and v[j] > 0: rs.append(math.log(raw/v[j]))
    if len(rs) < MARGIN_MIN_N: return None
    a = np.asarray(rs)
    return float(a.mean()), float(a.std()), len(rs)

PATTERN_MARGIN_SMALL = 1.55   # (legacy -- unused, kept for compatibility)
PATTERN_MARGIN_LARGE = 1.25
PATTERN_LARGE_SCALE  = 10_000_000

def _self_margin(scales, values):
    """
    [SELF-MARGIN] Measures the margin from the accuracy of earlier
    predictions **within the same series**: predict each scale from the
    one before it, record the ratio (predicted / actual), then take the
    worst undershoot and compensate for it.
```

```
    Why not used alone? Because it learns only from the past, so the first
    large jump catches it out. Measured on the actual series: coverage 8/9
    -- it failed at TRAIN_END=1,280,000 where the error was -34.7% while
    the learned margin was only 1.10 (because every prior error had been
    positive).
    It is therefore combined with a fixed floor: max(self, floor).
    """
    s = np.asarray(scales, dtype=float); v = np.asarray(values, dtype=float)
    ok = v > 0; s, v = s[ok], v[ok]
    if len(s) < 3: return None
    worst = 1.0; used = 0
    for i in range(2, len(s)):
        # [SCOPE-FIX] Restrict the measurement to jumps <= 2.5x -- the
        # model's actual operating range (training on [1,N] contains
        # [N+1,2N]).
        # Longer jumps within the series (e.g. 20,480,000 -> 5x10^11 =
        # 24,414x) are not operating cases, and including them spuriously
        # inflated the margin (1.93 instead of 1.39).
        if s[i]/s[i-1] > 2.5: continue
        k = math.log(v[i-1]/v[i-2]) / math.log(s[i-1]/s[i-2])
        raw = v[i-1] * (s[i]/s[i-1])**float(np.clip(k, 0.0, 1.0))
        worst = min(worst, raw/v[i]); used += 1
    if used < 3 or worst <= 0: return None
    return (1.0/worst) * 1.10

def predict_c_emp(scales, values, TRAIN_END):
    """
    Predicts C_EMP at TRAIN_END from a prior series of (scale, value).
    Method: a local exponent from the last two scales (more accurate than
    a global fit: mean error 16.1% versus 22.9%), multiplied by a margin.

    Margin = max(self-measured from the series, fixed floor).
    The floor guards against the first jump (where the self-measured value
    alone fails), while the self-measured value widens automatically if a
    jump larger than any observed appears. As the series lengthens and
    accuracy improves, the margin approaches its floor -- i.e. it
    **tightens automatically**.
    """
    s = np.asarray(scales, dtype=float); v = np.asarray(values, dtype=float)
    ok = v > 0; s, v = s[ok], v[ok]
    if len(s) < 2:
        raise ValueError("at least two actual scales are required")
    k = math.log(v[-1]/v[-2]) / math.log(s[-1]/s[-2])
    k = float(np.clip(k, 0.0, 1.0))
    raw = v[-1] * (float(TRAIN_END)/s[-1])**k
    # [MARGIN] The statistically derived margin, widened automatically if
    # the self-measured margin reveals an error larger than any seen
```

```
    # during derivation.
    # [MARGIN] mu and sigma are measured from the series itself on every
    # call, so any newly added direct calibration updates them
    # automatically.
    _mp = measure_margin_params(scales, values)
    derived = math.exp(MARGIN_Z * _mp[1] - _mp[0]) if _mp else MARGIN_FLOOR
    self_m = _self_margin(s, v)
    margin = max(MARGIN_FLOOR, derived, self_m or 0.0)
    return raw * margin, raw, k, margin

def calibration_from_pattern(TRAIN_END, P_ALL, scales, pn_values, pi_values,
                             verbose=True, grid=None, measure_c_emp=False):
    """
    Full calibration **without any sieving**: coefficients from a grid via
    primecount, and C_EMP by pattern prediction.

    [FIX] The first version called compute_calibration, which needs
    P_ALL -- so it failed exactly at the scales that cannot be sieved,
    which is the case mode F exists for. Now: P_ALL is used if available,
    otherwise the grid values are fetched from primecount (a few hundred
    thousand calls at most).
    """
    # [PERF] Grid size when P_ALL is absent: each point is a primecount
    # call, whose cost grows with TRAIN_END. At 2^50 a call takes about a
    # second, so 200K points would take days.
    # 20K points suffice for the coefficients (mean-type statistics --
    # verified that a grid is enough for them).
    if grid is None:
        if P_ALL is not None:
            grid = min(CALIB_GRID, 200_000)
        else:
            # [AUTO-GRID] Each point is a primecount call, whose cost
            # explodes with n:
            #   10¹² -> 222ms | 10¹³ -> 633ms | 10¹⁴ -> 2.1s | 2⁵⁰ -> 9.9s
            # A fixed size (20,000) meant 55 hours at 2^50 with no
            # warning at all.
            # Now: measure the cost of one call directly, and compute the
            # size that fits GRID_TIME_BUDGET -- so this cannot recur at
            # any scale.
            #
            # The floor of 500 is justified by measurement: the effect of
            # grid size on estimation error (TRAIN_END=5M) was 878 at 500
            # versus 1,040 at 200,000 -- i.e. the small grid is not worse
            # (the coefficients are mean-type statistics).
            _t0 = time.time()
            _m = _pc()
            if _m is not None:
```

```
            _m.nth_prime(int(TRAIN_END))
        _per = max(time.time() - _t0, 1e-6)
        grid = int(np.clip(GRID_TIME_BUDGET / _per, GRID_MIN, GRID_MAX))
        if verbose:
            _est = grid * _per
            print(f"  [auto-grid] one call={_per*1000:.0f}ms => "
                  f"grid {grid:,} points (~{_est/60:.0f} min)")
    grid = min(grid, TRAIN_END)
idx = np.unique(np.linspace(0, TRAIN_END-1, min(grid, TRAIN_END)).astype(np.i
n_g = (idx + 1).astype(np.float64)
if P_ALL is not None:
    p_g = P_ALL[idx].astype(np.float64)
else:
    m = _pc()
    if m is None:
        raise RuntimeError(
            "\n" + "="*62 + "\n"
            "primecount is not installed -- required for scales beyond the si
            "Install (one cell before running):\n"
            "    !pip install primecountpy\n\n"
            "then re-run this cell.\n" + "="*62)
    if verbose:
        print(f"  [pattern] fetching {len(idx):,} grid points via primecount.
    p_g = np.array([m.nth_prime(int(v)) for v in n_g], dtype=np.float64)

li_inv_g = inv_R_vec(n_g)
X = build_pn_features(li_inv_g)
y = p_g - li_inv_g
si = np.random.RandomState(42).choice(len(idx), min(15000, len(idx)), replace
al = RidgeCV(alphas=np.logspace(-1,3,30), fit_intercept=False, cv=3).fit(X[si
Rp = Ridge(alpha=al, fit_intercept=False, solver='cholesky').fit(X, y)
median_pn = float(np.median(y - Rp.predict(X)))
del X; gc.collect()

li_g = R_vec(p_g)
r0b = n_g - li_g
sm = savgol_filter(r0b, min(51, len(r0b)-(1-len(r0b)%2)), 3)
mk = (np.abs(r0b-sm) <= np.percentile(np.abs(r0b-sm), 70.4)) & (n_g >= 10)
def _bf(x,a,b): return a*np.sqrt(x/np.log(np.maximum(x,3.)))+b
(A_B, B_B), _ = curve_fit(_bf, p_g[mk], r0b[mk], maxfev=10000)
Fb_pi_g = li_g + _bf(p_g, A_B, B_B)
Xp = build_pi_features(p_g)
Ri = Ridge(alpha=al, fit_intercept=False, solver='cholesky').fit(Xp, n_g - Fb
median_pi = float(np.median((n_g - Fb_pi_g) - Ri.predict(Xp)))
del Xp; gc.collect()

# base slope from the grid (substitute for the convex hull when P_ALL
```

```
    # is unavailable)
    if P_ALL is not None:
        TSN = lower_hull_tsn_base(TRAIN_END, P_ALL)
    else:
        TSN = float(np.min(np.diff(p_g)/np.diff(n_g)))

    if measure_c_emp:
        # [C] Measure C_EMP by primecount sampling instead of predicting
        # -- more accurate but slower
        pn_pred = c_emp_via_primecount(TRAIN_END, Rp.coef_, median_pn, 'pn')
        pi_pred = c_emp_via_primecount(TRAIN_END, Ri.coef_, median_pi, 'pi',
                                       A_BIAS=A_B, B_BIAS=B_B)
        pn_raw, k_pn, m_ = pn_pred/PC_MARGIN, float('nan'), PC_MARGIN
    else:
        pn_pred, pn_raw, k_pn, m_ = predict_c_emp(scales, pn_values, TRAIN_END)
        pi_pred, pi_raw, k_pi, _  = predict_c_emp(scales, pi_values, TRAIN_END)

    def c_vec(a_): return np.zeros_like(np.asarray(a_, dtype=np.float64))
    cal_pn = dict(TRAIN_END=TRAIN_END, c_vec=c_vec, dc_dn=lambda n: 0.0,
                  ridge_coef_pn=Rp.coef_, DECAY=0.999, C_EMP_PN=pn_pred,
                  TSN_BASE=TSN, THRESH_PN=2.28*(TRAIN_END**0.486)*0.84,
                  median_corr_pn=median_pn, RHO=None, ANOM=None,
                  C_EMP_PREDICTED=True)
    cal_pi = dict(TRAIN_END=TRAIN_END, A_BIAS_PI=A_B, B_BIAS_PI=B_B,
                  ridge_coef=Ri.coef_, C_EMP_PI=pi_pred, DECAY_PI=0.999,
                  CP_PI=5, THRESH_PI=0.3, x_min=float(p_g[0]),
                  median_corr_pi=median_pi, C_EMP_PREDICTED=True)
    if verbose:
        _lbl = "measured" if measure_c_emp else "pattern"
        print(f"  [{_lbl} TRAIN_END={TRAIN_END:,}] C_EMP_PN={pn_pred:.3f} "
              f"(raw {pn_raw:.3f}, k={k_pn:.3f}) | C_EMP_PI={pi_pred:.4f} | margi
    return cal_pn, cal_pi


STREAM_CEMP = False   # True = full C_EMP scan without storage (coverage + bounde


USE_PRIMECOUNT = False      # True = obtain p(n) via primecount instead of sievin
PC_SAMPLE      = 100_000    # number of sample points for C_EMP
PC_MARGIN      = 1.10       # margin compensating the sampling shortfall -- justi
# Measured shortfall (C_EMP from a 100K sample / full C_EMP) across 5
# scales from 2M to 20M:
#     0.9780  0.9829  0.9532  0.9835  0.9917
#     min=0.9532   mean=0.9779   sd=0.0131
# Required margins:
#     to cover the worst measured : 1/0.9532 = 1.049
#     to cover (mean - 3 sigma)   : 1/0.9386 = 1.065
```

```
# We chose 1.10 -- wider than both with a safety gap, costing only 10% in
# width.
#
# **Warning: this is not a guarantee.** A method for computing the margin
# automatically from the sample itself was tried (a saturation indicator,
# C_EMP(50K)/C_EMP(100K)) and failed: its correlation with the actual
# shortfall was r=-0.32, with inverted sign. The shortfall is therefore
# **estimated statistically from prior measurements, not computed from
# the current run**.
# And C_EMP is a maximum-value statistic in any case, so it carries no
# guarantee -- the only proven component in the model is the Axler bound.

_PC = None
def _pc():
    """Loads primecount on first use; returns None if unavailable."""
    global _PC
    if _PC is None:
        try:
            from primecountpy import primecount as _m
            _PC = _m
        except ImportError:
            try:
                import primecount as _m
                _PC = _m
            except ImportError:
                _PC = False
    return _PC if _PC else None

def nth_prime_vec(ns):
    """p(n) for an array of indices, via primecount."""
    m = _pc()
    if m is None:
        raise RuntimeError("primecount unavailable -- install it: pip install pri
    return np.array([m.nth_prime(int(v)) for v in np.atleast_1d(ns)], dtype=np.fl

def c_emp_via_primecount(TRAIN_END, ridge_coef, median_corr, side='pn',
                         A_BIAS=None, B_BIAS=None, K=None):
    """
    Computes C_EMP over a spread sample, fetching p(n) from primecount --
    no sieve. The result is multiplied by PC_MARGIN to compensate for what
    the sample misses.
    """
    K = K or PC_SAMPLE
    ns = np.unique(np.linspace(C_EMP_MIN_ARG, TRAIN_END, int(min(K, TRAIN_END))).
    nf = ns.astype(np.float64)
    pv = nth_prime_vec(ns)
    if side == 'pn':
```

```
        bx = inv_R_vec(nf)
        F = bx + fast_ridge_correction(bx, G60_PN, ridge_coef) + median_corr
        ratio = np.abs(pv - F) / _NORM(nf)
    else:
        F = (R_vec(pv) + (A_BIAS*np.sqrt(pv/np.log(np.maximum(pv,3.)))+B_BIAS)
             + fast_ridge_correction(pv, G60_PI, ridge_coef) + median_corr)
        ratio = np.abs(nf - F) / _NORM(pv)
    return float(np.max(ratio)) * PC_MARGIN


# depending on the seed (measured: 3 of 4 targets give different bounds,
# differing by 1-3%). Taking the union over several spread seeds makes the
# result **independent of any particular choice**.

def inv_R_vec_fast(ns):
    """[PORT-5] Same as inv_R_vec but with the fast li seed -- used
    **inside the C_EMP loop only** (the one place that traverses millions
    of points). The walk and jump paths keep the original inv_R_vec
    unchanged."""
    ns = np.asarray(ns, dtype=float)
    x = inv_li_vec_fast_bulk(ns)
    x = x - D_corr(x)*np.log(x)
    lx = np.log(x)
    return x - ((li_vec(x) + D_corr(x)) - ns)*lx

ANOMALY_OFF = False         # [TEST-SWITCH] True = disable the anomaly mechanism

# [EXTRACT] How the interval is extracted from the candidate cloud.
# 0.0 = min/max over all candidates (original behaviour, **default**)
# >0  = percentile: trim EXTRACT_PCTL% from each tail before taking the
#       bounds.
#
# Why might it help? With min/max the result is set by the two extreme
# candidates alone, so any improvement in the cloud's **density** (from
# the anomaly mechanism or from branching) evaporates and never shows.
# With a percentile, density is what determines the result.
#
# Preliminary measurement (18 targets, TRAIN_END=2M):
#     min/max        : width 7,365  contained 18/18  min margin 21.7%
#     percentile 0.5%: width 7,032  contained 18/18  min margin 22.0%
#     percentile 2%  : width 6,578  contained 18/18  min margin 18.8%
#     percentile 5%  : width 6,003  contained 18/18  min margin 16.0%
#
# **Warning:** the percentile abandons coverage in principle (it trims
# tails deliberately), and 18 targets is a very small sample. Do not adopt
# it before measuring on hundreds of targets across at least 3 scales.
# The default 0.0 leaves behaviour unchanged.
```

```
EXTRACT_PCTL = 0.0

SAMPLE_CAP = 500_000
CALIB_GRID = 600_000
CP_PN = 40


# ============================================================
# [PORT-3] Parallel execution of the two C_EMP loops
# ============================================================
# C_EMP_PN and C_EMP_PI are entirely independent (each needs only P_ALL
# and its own coefficients), and they are the two most expensive items in
# calibration. Running them together in two processes saves about half.
# P_ALL is inherited by fork without copying (Linux/Colab). Under spawn we
# fall back to sequential execution.
# Note: **numbers** are passed, not closures -- closures are not
# picklable.
_MP_P = None

def _mp_cemp(args):
    side, TE, prm = args
    P_ALL = _MP_P
    best = 0.0
    for cs in range(0, TE, C_EMP_CHUNK):
        ce = min(cs + C_EMP_CHUNK, TE)
        if side == 'pn':
            arg = np.arange(cs+1, ce+1, dtype=np.float64)
            b = inv_R_vec_fast(arg)
            F = b + fast_ridge_correction(b, G60_PN, prm['rc']) + prm['med']
            tgt = P_ALL[cs:ce]; den = arg
        else:
            arg = P_ALL[cs:ce].astype(np.float64)
            F = (R_vec(arg) + (prm['A']*np.sqrt(arg/np.log(np.maximum(arg,3.)))+p
                 + fast_ridge_correction(arg, G60_PI, prm['rc']) + prm['med'])
            tgt = np.arange(cs+1, ce+1, dtype=np.float64); den = arg
        k = den >= 10_000
        if k.any():
            m = float(np.max(np.abs(tgt[k] - F[k]) / _NORM(den[k])))
            if m > best: best = m
        del arg, F, tgt, den, k
        if cs % (C_EMP_CHUNK*10) == 0: gc.collect()
    return best

def _cemp_pair(TRAIN_END, P_ALL, pn_prm, pi_prm, parallel):
    global _MP_P
    _ensure_D()   # [FIX] built once in the parent before fork, so both
                  # children inherit it without rebuilding
```

```
    if parallel:
        import multiprocessing as _mp
        if _mp.get_start_method(allow_none=True) in ('fork', None):
            try:
                import concurrent.futures as cf
                print("  [C_EMP: running in parallel on two processes]")
                _MP_P = P_ALL
                with cf.ProcessPoolExecutor(max_workers=2) as ex:
                    f1 = ex.submit(_mp_cemp, ('pn', TRAIN_END, pn_prm))
                    f2 = ex.submit(_mp_cemp, ('pi', TRAIN_END, pi_prm))
                    r = (f1.result(), f2.result())
                _MP_P = None
                return r
            except Exception as _e:
                print(f"  [parallel failed: {_e} -- falling back to sequential]")
                _MP_P = None
    if parallel: print("  [parallel unavailable (spawn) -- sequential]")
    _MP_P = P_ALL
    r = (_mp_cemp(('pn', TRAIN_END, pn_prm)), _mp_cemp(('pi', TRAIN_END, pi_prm))
    _MP_P = None
    return r



# ============================================================
# [CACHE] Saving and reusing a calibration
# ============================================================
# Calibration costs 2,388s at TRAIN_END=500M, whereas a single jump takes
# 11s. Saving it once and reloading makes any later target from the same
# TRAIN_END almost instantaneous.
# Note: c_vec and dc_dn are functions (not picklable), so we store their
# parameters and rebuild them on load -- the result is **bit-identical**.
def save_calibration(path, cal_pn, cal_pi):
    d_pn = {k: v for k, v in cal_pn.items() if k not in ('c_vec', 'dc_dn')}
    with open(path, 'wb') as f:
        pickle.dump({'pn': d_pn, 'pi': cal_pi, 'ver': 'ln2-R-noc'}, f)

def load_calibration(path):
    with open(path, 'rb') as f:
        d = pickle.load(f)
    cal_pn = d['pn']
    # c(n) is removed in this version => both functions are zero (see [C-REMOVED]
    cal_pn['c_vec'] = lambda n_arr: np.zeros_like(np.asarray(n_arr, dtype=np.floa
    cal_pn['dc_dn'] = lambda n: 0.0
    return cal_pn, d['pi']
```

```
def compute_calibration(TRAIN_END, P_ALL, verbose=True, parallel=False):
    t_start = time.time()
    rng = np.random.RandomState(42)
    grid_n = min(CALIB_GRID, TRAIN_END)
    idx_grid = np.unique(np.linspace(0, TRAIN_END-1, grid_n).astype(np.int64))
    n_g = (idx_grid + 1).astype(np.float64)
    p_g = P_ALL[idx_grid].astype(np.float64)
    li_inv_g = inv_R_vec(n_g)          # [UPGRADE-R]

    r0 = p_g - li_inv_g
    r0_sm = savgol_filter(r0, min(51, len(r0)-(1-len(r0)%2)), 3)
    mask = (np.abs(r0-r0_sm) <= np.percentile(np.abs(r0-r0_sm), 70.4)) & (n_g>=10
    # [C-REMOVED] c(n)=a*sqrt(n*ln n)+b corrected the bias of li^-1. After
    # upgrading the base to R^-1 that bias vanished, so it began fitting
    # noise.
    # Measured on 7,997 out-of-training targets (with full Ridge refit):
    #     with c(n)   : mean error 599   median 532
    #     without c(n): mean error 554   median 480   (8.15% better, p=1e-47)
    # Its purpose (centring the model on the sample) is served more
    # accurately by median_corr -- which is computed over **all** points
    # after Ridge, not on a partial 70.4% mask.
    a_c, b_c = 0.0, 0.0
    def c_vec(n_arr): return np.zeros_like(np.asarray(n_arr, dtype=np.float64))
    def dc_dn(n): return 0.0
    del r0, r0_sm, mask; gc.collect()

    samp_idx = rng.choice(len(idx_grid), min(SAMPLE_CAP, len(idx_grid)), replace=
    Fb_pn_samp = li_inv_g[samp_idx] + c_vec(n_g[samp_idx])
    y_pn = p_g[samp_idx] - Fb_pn_samp
    Xs_pn = build_pn_features(li_inv_g[samp_idx])
    si2 = rng.choice(len(samp_idx), min(15000,len(samp_idx)), replace=False)
    alpha_pn = RidgeCV(alphas=np.logspace(-1,3,30), fit_intercept=False, cv=3).fi
    Rdg_pn = Ridge(alpha=alpha_pn, fit_intercept=False, solver='cholesky').fit(Xs
    ridge_coef_pn = Rdg_pn.coef_
    del Xs_pn, y_pn; gc.collect()

    Fb_pn_grid = li_inv_g + c_vec(n_g)
    Xg_pn = build_pn_features(li_inv_g)
    base_pred_pn = Rdg_pn.predict(Xg_pn)
    median_pn = float(np.median((p_g - Fb_pn_grid) - base_pred_pn))
    del Fb_pn_grid, Xg_pn, base_pred_pn
    # [FIX-2] It was n_g >= 10. Measured at TRAIN_END=2,000,000:
    #   cutoff n>=10     -> C_EMP=64.250, worst point at n=11 (!)
    #   cutoff n>=100    -> C_EMP=16.800, worst point at n=111
    #   cutoff n>=10,000 -> C_EMP= 7.553, worst point at n=1,887,313
    #                       (a genuine gap)
    # Reason: ln^2(11)=5.75 is a trivially small denominator, so dividing
```

```
# by it inflates the ratio spuriously.
# The value is entirely stable from n>=10,000 upward. Impact: C_EMP
# inflated 8.5x.
# [NORM] Normalisation: ln^2(n) is borrowed from Cramer's conjecture
# on the largest prime gap, but C_EMP measures the **model's** residual,
# not gaps. Measured across 4 bins (dispersion = max/min; ideal 1.0):
#     ln^2(n) [current] : 1.57x
#     √n·ln(n)        : 1.21x
#     n^0.4           : 1.11x
#     √n              : 1.10x
#     sqrt(n)/ln(n)     : 1.08x   <- most uniform
# sqrt(n)/ln(n) is also the form the Riemann hypothesis suggests for
# the size of pi(x) fluctuations, so it is not a purely empirical
# choice. It was nevertheless rejected: see the [NORM] note above --
# it widens the interval 1.8x under extrapolation.
# [PORT-1] C_EMP over the **entire** training range (every point, in
# chunks) instead of a fixed 600K grid. C_EMP is a **maximum-value**
# statistic, so a sample can miss the true extreme. Chunking is
# necessary for memory (a single batch at 50M would OOM).
del Fb_pn_samp; gc.collect()
if STREAM_CEMP:
    # [STREAM] full scan without storage -- full coverage, bounded
    # memory, no margin needed
    C_EMP_PN = c_emp_streaming(TRAIN_END, ridge_coef_pn, median_pn, 'pn')
elif USE_PRIMECOUNT and _pc() is not None:
    # [PRIMECOUNT] sampling without a sieve -- higher ceiling, but
    # with an estimated margin
    C_EMP_PN = c_emp_via_primecount(TRAIN_END, ridge_coef_pn, median_pn, 'pn'
_PN_PRM = dict(rc=ridge_coef_pn, med=median_pn)   # [PORT-3] computed later,
if not (STREAM_CEMP or (USE_PRIMECOUNT and _pc() is not None)):
    C_EMP_PN = None

# [FIX-3] It used to take a 3000-point sample of the checkpoints and
# then compute the correlation between them. But after subsampling,
# the spacing between adjacent points became (range/3000), not CP_PN.
# At TRAIN_END=50M this measures a lag of ~16,680 instead of 40 -- an
# entirely wrong lag.
# Proved with a synthetic AR(1) series of known correlation 0.85:
#     old method    -> -0.0355  (error 0.886)
#     block method  -> +0.849   (error 0.001)
# Fix: short blocks of **consecutive** points at the true CP spacing,
# distributed over the range, preserving lag=CP while still covering
# the whole range.
_nb, _bl = 300, min(40, max(3, TRAIN_END // (CP_PN * 6)))
_span_end = max(CP_PN + 1.0, TRAIN_END - CP_PN*_bl - 1)
_anchors = np.linspace(CP_PN, _span_end, min(_nb, max(2, int(_span_end/CP_PN)
_pts = np.empty(len(_anchors)*_bl, dtype=np.float64); _i = 0
```

```
for _a in _anchors:
    _c = float(_a)
    for _ in range(_bl):
        _c += CP_PN; _pts[_i] = _c; _i += 1
_true = P_ALL[(_pts-1).astype(np.int64)].astype(np.float64)
_model = inv_R_vec(_pts) + c_vec(_pts)   # [UPGRADE-R]
_rb = (_true - _model).reshape(len(_anchors), _bl)
_a1 = _rb[:, :-1].ravel(); _a2 = _rb[:, 1:].ravel()
DECAY_pn = float(np.corrcoef(_a1, _a2)[0,1]) if len(_a1) > 2 else 0.5
del _pts, _true, _model, _rb, _a1, _a2; gc.collect()

# [ONLINE] online-mode measurements -- cheap (a few thousand points)
_cl_on = dict(c_vec=c_vec, ridge_coef_pn=ridge_coef_pn, median_corr_pn=median
try:
    RHO  = measure_residual_rho(TRAIN_END, P_ALL, _cl_on)
    ANOM = measure_anomaly_persistence(TRAIN_END, P_ALL, _cl_on)
except Exception as _e:
    print(f'  [ONLINE] RHO/ANOM measurement failed: {_e}')
    RHO, ANOM = None, None
TSN_BASE = lower_hull_tsn_base(TRAIN_END, P_ALL)
THRESH_PN = 2.28*(TRAIN_END**0.486)*0.84
cal_pn = dict(TRAIN_END=TRAIN_END, c_vec=c_vec, dc_dn=dc_dn, ridge_coef_pn=ri
              DECAY=DECAY_pn, C_EMP_PN=C_EMP_PN, TSN_BASE=TSN_BASE, THRESH_PN
              median_corr_pn=median_pn, RHO=RHO, ANOM=ANOM)

li_g = R_vec(p_g)                  # [UPGRADE-R]
r0b = n_g - li_g
r0b_sm = savgol_filter(r0b, min(51, len(r0b)-(1-len(r0b)%2)), 3)
maskb = (np.abs(r0b-r0b_sm) <= np.percentile(np.abs(r0b-r0b_sm), 70.4)) & (n_
def _bf(x,a,b): return a*np.sqrt(x/np.log(np.maximum(x,3.)))+b
(A_BIAS_PI,B_BIAS_PI),_ = curve_fit(_bf, p_g[maskb], r0b[maskb], maxfev=10000
Fb_pi_g = li_g + (A_BIAS_PI*np.sqrt(p_g/np.log(np.maximum(p_g,3.)))+B_BIAS_PI
del r0b, r0b_sm, maskb; gc.collect()

samp_idx2 = rng.choice(len(idx_grid), min(SAMPLE_CAP, len(idx_grid)), replace
Xs_pi = build_pi_features(p_g[samp_idx2])
y_pi = (n_g - Fb_pi_g)[samp_idx2]
si3 = rng.choice(len(samp_idx2), min(15000,len(samp_idx2)), replace=False)
alpha_pi = RidgeCV(alphas=np.logspace(-1,3,30), fit_intercept=False, cv=3).fi
Rdg_pi = Ridge(alpha=alpha_pi, fit_intercept=False, solver='cholesky').fit(Xs
del Xs_pi, y_pi; gc.collect()

Xg_pi = build_pi_features(p_g)
base_pred_pi = Rdg_pi.predict(Xg_pi)
median_pi = float(np.median((n_g - Fb_pi_g) - base_pred_pi))
del Xg_pi, base_pred_pi, Fb_pi_g, li_g, idx_grid, n_g, p_g, li_inv_g; gc.coll
# [PORT-1] same principle for the pi(x) side
```

```
_PI_PRM = dict(rc=Rdg_pi.coef_, med=median_pi, A=A_BIAS_PI, B=B_BIAS_PI)
if STREAM_CEMP:
    C_EMP_PI = c_emp_streaming(TRAIN_END, Rdg_pi.coef_, median_pi, 'pi',
                               A_BIAS=A_BIAS_PI, B_BIAS=B_BIAS_PI)
elif USE_PRIMECOUNT and _pc() is not None:
    C_EMP_PI = c_emp_via_primecount(TRAIN_END, Rdg_pi.coef_, median_pi, 'pi',
                                    A_BIAS=A_BIAS_PI, B_BIAS=B_BIAS_PI)
else:
    C_EMP_PN, C_EMP_PI = _cemp_pair(TRAIN_END, P_ALL, _PN_PRM, _PI_PRM, paral
cal_pn['C_EMP_PN'] = C_EMP_PN
gc.collect()

CP_PI = 5
# [FIX-CONSISTENCY] It used to walk **every** checkpoint from the
# first prime to the end of training -- 10 million points at
# TRAIN_END=50M, with a full build_pi_features for them. It was also
# **inconsistent** with DECAY_PN, which had been fixed to the block
# method (since a spread-out sample measures the wrong lag).
# Both sides now use the same method: short blocks of consecutive
# points at the true CP spacing, distributed over the range.
# Measured at TRAIN_END=5M:
#     full walk : DECAY_PI=0.99846695  in 3.67s (1,000,138 points)
#     blocks    : DECAY_PI=0.99849368  in 0.04s (12,000 points)
#     difference 2.7e-05, 84x faster
_nbp, _blp = 300, 40
_anch_pi = inv_R_vec(np.linspace(CP_PN, TRAIN_END-1, _nbp))
_pts_pi = np.empty(_nbp*_blp, dtype=np.float64); _j = 0
for _aa in _anch_pi:
    _c = float(_aa)
    for _ in range(_blp):
        _c += CP_PI*math.log(max(_c, 3.)); _pts_pi[_j] = _c; _j += 1
# [COMPACT-FIX] It used searchsorted over P_ALL[:TRAIN_END], which
# materialises the whole array (8 GB at 10^9). Now each point is
# searched only within the anchor block it falls in, so only small
# slices are reconstructed.
if isinstance(P_ALL, CompactPrimes):
    _tk = np.empty(len(_pts_pi), dtype=np.float64)
    _ord = np.argsort(_pts_pi); _sp = _pts_pi[_ord]
    _ai = np.searchsorted(P_ALL.anchors, _sp, side='right') - 1
    _ai = np.clip(_ai, 0, len(P_ALL.anchors)-1)
    _i = 0
    while _i < len(_sp):
        _blk = _ai[_i]; _j = _i
        while _j < len(_sp) and _ai[_j] == _blk: _j += 1
        _lo = 1 + _blk*ANCHOR_STRIDE
        _hi = min(_lo + ANCHOR_STRIDE + 1, TRAIN_END)
        _seg = P_ALL[_lo:_hi]
```

```
            # [EDGE-FIX] a point beyond the last prime in range used to
            # return TRAIN_END+1; we clip it at TRAIN_END to match the
            # original searchsorted.
            _tk[_ord[_i:_j]] = np.minimum(_lo + np.searchsorted(_seg, _sp[_i:_j],
            _i = _j; del _seg
    else:
        _tk = np.searchsorted(P_ALL[:TRAIN_END], _pts_pi, side='right').astype(np
    _fz = ((A_BIAS_PI*np.sqrt(_pts_pi/np.log(np.maximum(_pts_pi,3.)))+B_BIAS_PI)
           + R_vec(_pts_pi) + build_pi_features(_pts_pi).dot(Rdg_pi.coef_))
    _rbp = (_tk - _fz).reshape(_nbp, _blp)
    DECAY_PI = float(np.corrcoef(_rbp[:,:-1].ravel(), _rbp[:,1:].ravel())[0,1])
    del _pts_pi, _tk, _fz, _rbp; gc.collect()

    cal_pi = dict(TRAIN_END=TRAIN_END, A_BIAS_PI=A_BIAS_PI, B_BIAS_PI=B_BIAS_PI,
                  ridge_coef=Rdg_pi.coef_, C_EMP_PI=C_EMP_PI, DECAY_PI=DECAY_PI,
                  CP_PI=CP_PI, THRESH_PI=0.3, x_min=float(P_ALL[0]),
                  median_corr_pi=median_pi)
    if verbose:
        print(f"  [calib TRAIN_END={TRAIN_END:,}] C_EMP_PN={C_EMP_PN:.3f} DECAY_P
              f"C_EMP_PI={C_EMP_PI:.4f} DECAY_PI={DECAY_PI:.8f}  ({time.time()-t_
    return cal_pn, cal_pi

# ============================================================
# 7. jump_walk_pi -- independent memory
# ============================================================
def jump_walk_pi(x_start, x_end_target, cal_pi, off0=None, sf0=None, wo0=None,
                  MX_PI=1024, SEL_PI=512, PS_SCALE_PI=7,
                  CONF_THRESH_PI=None, PHASE1_CPS_PI=6, PHASE3_CPS_PI=6, ANOM_DEC
    if CONF_THRESH_PI is None: CONF_THRESH_PI = 1e9 if ANOMALY_OFF else 0.08   #
    A_BIAS_PI=cal_pi['A_BIAS_PI']; B_BIAS_PI=cal_pi['B_BIAS_PI']; ridge_coef=cal_
    CP_PI=cal_pi['CP_PI']; C_EMP_PI=cal_pi['C_EMP_PI']; DECAY_PI=cal_pi['DECAY_PI
    THRESH_PI=cal_pi['THRESH_PI']
    median_corr_pi = cal_pi.get('median_corr_pi', 0.0)

    x_cur = x_start
    off = np.zeros(1, float) if off0 is None else off0.copy()
    sf  = np.zeros(1, float) if sf0  is None else sf0.copy()
    cb  = np.zeros(1, int); ch = np.zeros(1, int)

    if wo0 is None:
        wo = np.full(len(off), (Fz_pi(x_start, A_BIAS_PI, B_BIAS_PI, ridge_coef)
                                - Fb_pi(x_start, A_BIAS_PI, B_BIAS_PI)
                                + median_corr_pi))
    else:
        wo = wo0.copy()

    anom_phase = np.zeros(len(off), dtype=int)
```

```
anom_dir   = np.zeros(len(off), dtype=int)
anom_amp   = np.zeros(len(off), dtype=float)
anom_ph_cps= np.zeros(len(off), dtype=int)

snapshots = []
step = 0
_max_steps = int(3 * max(x_end_target - x_cur, 1) / (CP_PI * math.log(max(x_c

while x_cur < x_end_target and step < _max_steps:
    step += 1
    ln_x = math.log(x_cur)
    x_new = x_cur + CP_PI * ln_x
    if x_new > x_end_target: break

    Fb = Fb_pi(x_new, A_BIAS_PI, B_BIAS_PI)
    w = DECAY_PI * wo * math.exp(-1.0)
    vals = Fb + w + off + sf * CP_PI
    max_jump = C_EMP_PI * _NORM_S(x_cur)

    ae = np.where(anom_phase == 1, anom_dir * anom_amp,
                  np.where(anom_phase == 3, -anom_dir * anom_amp * ANOM_DECAY

    snapshots.append((x_cur, x_new, off.astype(np.float32), sf.astype(np.floa

    tsn_pi = 1.0 / ln_x
    dFb_dx = (1.0 / ln_x) + dbias_pi_dx(x_new, A_BIAS_PI, B_BIAS_PI)
    dWn = DECAY_PI * wo * (-1.0 / (CP_PI * ln_x)) * math.exp(-1.0)
    dfb = dFb_dx + sf + dWn
    dev = np.abs(dfb - tsn_pi) / max(abs(tsn_pi), 1e-12)

    th  = np.full(len(off), 0.02)
    cf_ = np.full(len(off), 0.60)
    m3 = cb >= 3
    if m3.any():
        mv = np.minimum(cb[m3], 5)
        th[m3] = np.minimum(0.05, 0.02 * (1.5 ** mv))
        cf_[m3] = np.maximum(0.30, 0.60 * (0.9 ** cb[m3]))
    m5 = ch >= 5
    if m5.any(): th[m5] = np.maximum(0.005, th[m5] * (0.85 ** ch[m5]))

    ff = np.clip((tsn_pi - dfb) * cf_ * CP_PI, -1.5, 1.5)
    ps = np.abs(ff) * PS_SCALE_PI
    sn = np.clip(sf + ff, -1.5, 1.5)
    sr = np.clip(-ff, -1.5, 1.5)

    is_hold = dev <= th
    is_brake = ~is_hold & (vals > Fb + THRESH_PI)
```

```
is_jump = ~is_hold & ~is_brake & (vals < Fb - THRESH_PI)

no, ns_, nc, nh = [], [], [], []
nw = []
na_phase, na_dir, na_amp, na_ph = [], [], [], []

for j in range(len(off)):
    o, s_, c_, h_ = off[j], sf[j], int(cb[j]), int(ch[j])
    f_, p_, sn_, sr_ = ff[j], ps[j], sn[j], sr[j]
    wj = wo[j]
    ap, ad, aa, apc = anom_phase[j], anom_dir[j], anom_amp[j], anom_ph_cp

    if is_hold[j]:
        no.append(o); ns_.append(f_ * 0.3); nc.append(0); nh.append(h_ +
        nw.append(wj)
        na_phase.append(ap); na_dir.append(ad); na_amp.append(aa); na_ph.
    elif is_brake[j]:
        no.append(o - p_); ns_.append(sn_); nc.append(c_ + 1); nh.append(
        nw.append(wj)
        na_phase.append(ap); na_dir.append(ad); na_amp.append(aa); na_ph.
    elif is_jump[j]:
        no.append(o + p_); ns_.append(sn_); nc.append(c_ + 1); nh.append(
        nw.append(wj)
        na_phase.append(ap); na_dir.append(ad); na_amp.append(aa); na_ph.
    else:
        o3 = (o + p_) if dfb[j] > tsn_pi else (o - p_)
        no.extend([o + p_, o - p_, o3])
        ns_.extend([sn_, sn_, sr_])
        nc.extend([c_ + 1] * 3)
        nh.extend([0] * 3)
        nw.extend([wj, wj, wj])
        na_phase.extend([ap, ap, ap])
        na_dir.extend([ad, ad, ad])
        na_amp.extend([aa, aa, aa])
        na_ph.extend([apc, apc, apc])

off = np.array(no, float)
sf  = np.array(ns_, float)
cb  = np.array(nc, int)
ch  = np.array(nh, int)
wo  = np.array(nw, float)
anom_phase = np.array(na_phase, int)
anom_dir   = np.array(na_dir, int)
anom_amp   = np.array(na_amp, float)
anom_ph_cps= np.array(na_ph, int)

if len(off) > MX_PI:
```

```
            idx = np.argsort(off, kind='stable')
            keep = np.round(np.linspace(0, len(idx) - 1, SEL_PI)).astype(int)
            off = off[idx[keep]]; sf = sf[idx[keep]]
            cb  = cb[idx[keep]];  ch = ch[idx[keep]]
            wo  = wo[idx[keep]]
            anom_phase = anom_phase[idx[keep]]
            anom_dir   = anom_dir[idx[keep]]
            anom_amp   = anom_amp[idx[keep]]
            anom_ph_cps= anom_ph_cps[idx[keep]]

        if len(off):
            off -= off.mean()

        target_wo = (Fz_pi(x_new, A_BIAS_PI, B_BIAS_PI, ridge_coef) - Fb) + media
        wo = DECAY_PI * wo + (1.0 - DECAY_PI) * target_wo

        confidence = np.abs(wo) / (C_EMP_PI * _NORM_S(x_cur) + 1e-12)
        for j in range(len(off)):
            if anom_phase[j] == 0:
                if confidence[j] >= CONF_THRESH_PI:
                    anom_phase[j] = 1
                    anom_dir[j] = 1 if wo[j] > 0 else -1
                    anom_amp[j] = abs(wo[j])
                    anom_ph_cps[j] = 1
            elif anom_phase[j] == 1:
                anom_ph_cps[j] += 1
                if anom_ph_cps[j] > PHASE1_CPS_PI:
                    anom_phase[j] = 2
                    anom_ph_cps[j] = 1
            elif anom_phase[j] == 2:
                anom_phase[j] = 3
                anom_ph_cps[j] = 1
            elif anom_phase[j] == 3:
                anom_ph_cps[j] += 1
                if anom_ph_cps[j] > PHASE3_CPS_PI:
                    anom_phase[j] = 0
                    anom_dir[j] = 0
                    anom_amp[j] = 0.0
                    anom_ph_cps[j] = 0

        x_cur = x_new
    return snapshots

# ============================================================
# 8. make_pi_cloud_fn
# ============================================================
def make_pi_cloud_fn(snaps, cal_pi):
```

```
    if len(snaps)==0: return lambda x: (float('inf'), -float('inf'))
    PI_SNAP_X = np.array([s[1] for s in snaps])
    A_BIAS_PI=cal_pi['A_BIAS_PI']; B_BIAS_PI=cal_pi['B_BIAS_PI']
    C_EMP_PI=cal_pi['C_EMP_PI']; DECAY_PI=cal_pi['DECAY_PI']; CP_PI=cal_pi['CP_PI
    @lru_cache(maxsize=200000)
    def _cached(x_query):
        if x_query < PI_SNAP_X[0] or x_query > PI_SNAP_X[-1]:
            return (float('inf'), -float('inf'))
        idx=int(np.searchsorted(PI_SNAP_X,x_query)); idx=min(max(idx,0),len(snaps
        x_prev,x_new,off_s,sf_s,wo_s,ae_s = snaps[idx]
        if x_query < x_prev: return (float('inf'), -float('inf'))
        frac=(x_query-x_prev)/max(x_new-x_prev,1e-9); frac=min(max(frac,0.0),1.0)
        delta_k=frac*CP_PI; ln_xp=math.log(max(x_prev,3.))
        Fb_q=Fb_pi(x_query,A_BIAS_PI,B_BIAS_PI)
        w_q=DECAY_PI*wo_s*math.exp(-delta_k/CP_PI)
        vals_q=Fb_q+w_q+off_s+sf_s*delta_k
        max_jump_q=C_EMP_PI*_NORM_S(x_prev)
        expanded=[vals_q]
        for m in (1.0,1.5,2.0):
            expanded.append(vals_q+m*max_jump_q)
            expanded.append(vals_q-m*max_jump_q)
        if np.max(np.abs(ae_s))>0.5:
            expanded.append(vals_q+ae_s)
            expanded.append(vals_q-ae_s)
        full=np.concatenate(expanded)
        return (float(full.min()), float(full.max()))
    return _cached

def find_boundary(n, x_center, direction, cloud_fn, x_step_init=50, max_expand=50
                  max_extra_steps=500, x_limit=None):
    x_out=None; step=x_step_init; expand_count=0; last_inside=x_center
    def _past_limit(x):
        return x_limit is not None and ((direction>0 and x>x_limit) or (direction
    while expand_count<max_expand:
        x_try=x_center+direction*step
        if _past_limit(x_try): return last_inside, False
        if x_try<=2.0: break
        cmin,cmax=cloud_fn(x_try)
        if cmin<=n<=cmax: last_inside=x_try; step*=1.5; expand_count+=1
        else:
            if x_out is None: x_out=x_try
            step*=1.5; expand_count+=1; break
    if x_out is None: return last_inside, False
    extra_step=step; x_current=x_out; found_again=False
    for _ in range(max_extra_steps):
        x_try=x_current+direction*extra_step
        if _past_limit(x_try): break
```

```
        if x_try<=2.0: break
        cmin,cmax=cloud_fn(x_try)
        if cmin<=n<=cmax: x_after_gap=x_try; found_again=True; break
        x_current=x_try; extra_step*=1.5
    if found_again:
        x1,x2=x_out,x_after_gap
        for _ in range(40):
            mid=(x1+x2)/2; cmin,cmax=cloud_fn(mid)
            if cmin<=n<=cmax: x2=mid
            else: x1=mid
        return x2,True
    else:
        x1,x2=last_inside,x_out
        for _ in range(40):
            mid=(x1+x2)/2; cmin,cmax=cloud_fn(mid)
            if cmin<=n<=cmax: x1=mid
            else: x2=mid
        return x1,False

# ═══════════════════════════════════════════════════════════
# 9. jump_walk_pn -- independent memory + dimension fix
# ═══════════════════════════════════════════════════════════
def jump_walk_pn(n_start, n_end_target, cal_pn, off0=None, sf0=None):
    CP, MX, SEL = 40, 128, 64
    c_vec=cal_pn['c_vec']; DECAY=cal_pn['DECAY']; C_EMP_PN=cal_pn['C_EMP_PN']
    THRESH_PN=cal_pn['THRESH_PN']; TSN_BASE=cal_pn['TSN_BASE']; TRAIN_END=cal_pn[
    ridge_coef_pn = cal_pn['ridge_coef_pn']; dc_dn_fn = cal_pn['dc_dn']
    median_corr_pn = cal_pn.get('median_corr_pn', 0.0)

    off=np.zeros(1,float) if off0 is None else off0.copy()
    sf=np.zeros(1,float) if sf0 is None else sf0.copy()
    cb=np.zeros(1,int); ch=np.zeros(1,int)

    li_start = inv_R_scalar(float(n_start))  # [UPGRADE-R]
    feat_start = build_pn_features(np.array([li_start]))[0]
    wo = np.full(len(off), float(np.dot(feat_start, ridge_coef_pn)) + median_corr

    anom_phase = np.zeros(len(off), dtype=int)
    anom_dir   = np.zeros(len(off), dtype=int)
    anom_amp   = np.zeros(len(off), dtype=float)
    anom_ph_cps= np.zeros(len(off), dtype=int)

    lcp=n_start
    # [TEST-SWITCH] ANOMALY_OFF=True disables the anomaly mechanism to
    # measure its effect. Default False -- i.e. model behaviour
    # unchanged.
    PHASE1_CPS,PHASE3_CPS,CONF_THRESH = 2,2,(1e9 if ANOMALY_OFF else 0.30)
```

```
av_hist=[]
n_range = np.arange(n_start+1, n_end_target+1, dtype=np.float64)
li_range = inv_R_vec(n_range)            # [UPGRADE-R]
c_range = c_vec(n_range)
n = n_start
for _i in range(len(n_range)):
    n += 1
    li_n = li_range[_i]
    Fb = li_n + c_range[_i]
    delta=float(n-lcp)
    w = DECAY * wo * math.exp(-delta/CP)
    vals = Fb + w + off + sf*delta

    ln_n=math.log(n); max_jump=C_EMP_PN*_NORM_S(n)
    ae = np.where(anom_phase==1, anom_dir*anom_amp,
                  np.where(anom_phase==3, -anom_dir*anom_amp*0.4, 0.0))

    candidates = []
    for j in range(len(off)):
        base = Fb + w[j] + off[j] + sf[j]*delta
        candidates.append(base)
        for m in (1.0, 1.5, 2.0):
            candidates.append(base + m*max_jump)
            candidates.append(base - m*max_jump)
        if abs(ae[j]) > 1.0:
            candidates.append(base + ae[j])
            candidates.append(base - ae[j])
    av = np.array(candidates, dtype=float)
    av_hist.append((n, av.copy()))

    if n % CP == 0:
        tsn=TSN_BASE+(n-TRAIN_END)/TRAIN_END
        dWn=DECAY*wo*(-1.0/CP)*math.exp(-delta/CP)
        dfb = math.log(li_n) + dc_dn_fn(n) + sf + dWn
        dev=np.abs(dfb-tsn)/max(abs(tsn),1e-9)
        th=np.full(len(off),0.02); cf_=np.full(len(off),0.60)
        m3=cb>=3
        if m3.any():
            mv=np.minimum(cb[m3],5); th[m3]=np.minimum(0.05,0.02*(1.5**mv));
        m5=ch>=5
        if m5.any(): th[m5]=np.maximum(0.005,th[m5]*(0.85**ch[m5]))
        ff=np.clip((tsn-dfb)*cf_,-1.5,1.5); ps=np.abs(ff)*50
        sn=np.clip(sf+ff,-1.5,1.5); sr=np.clip(-ff,-1.5,1.5)
        is_hold=(dev<=th); is_brake=~is_hold&(vals>Fb+THRESH_PN); is_jump=~is

        no,ns_,nc,nh=[],[],[],[]
        nw=[]
```

```
na_phase,na_dir,na_amp,na_ph=[],[],[],[]
for j in range(len(off)):
    o,s_,c_,h_=off[j],sf[j],int(cb[j]),int(ch[j])
    f_,p_,sn_,sr_=ff[j],ps[j],sn[j],sr[j]
    wj=wo[j]
    ap,ad,aa,apc=anom_phase[j],anom_dir[j],anom_amp[j],anom_ph_cps[j]
    if is_hold[j]:
        no.append(o); ns_.append(f_*0.3); nc.append(0); nh.append(h_+
        nw.append(wj)
        na_phase.append(ap); na_dir.append(ad); na_amp.append(aa); na
    elif is_brake[j]:
        no.append(o-p_); ns_.append(sn_); nc.append(c_+1); nh.append(
        nw.append(wj)
        na_phase.append(ap); na_dir.append(ad); na_amp.append(aa); na
    elif is_jump[j]:
        no.append(o+p_); ns_.append(sn_); nc.append(c_+1); nh.append(
        nw.append(wj)
        na_phase.append(ap); na_dir.append(ad); na_amp.append(aa); na
    else:
        o3=(o+p_) if dfb[j]>tsn else (o-p_)
        no.extend([o+p_,o-p_,o3]); ns_.extend([sn_,sn_,sr_])
        nc.extend([c_+1]*3); nh.extend([0]*3)
        nw.extend([wj,wj,wj])
        na_phase.extend([ap,ap,ap]); na_dir.extend([ad,ad,ad])
        na_amp.extend([aa,aa,aa]); na_ph.extend([apc,apc,apc])

off=np.array(no,float); sf=np.array(ns_,float)
cb=np.array(nc,int); ch=np.array(nh,int)
wo=np.array(nw,float)
anom_phase=np.array(na_phase,int); anom_dir=np.array(na_dir,int)
anom_amp=np.array(na_amp,float); anom_ph_cps=np.array(na_ph,int)

if len(off)>MX:
    idx=np.argsort(off); ki=np.round(np.linspace(0,len(idx)-1,SEL)).a
    off=off[idx[ki]]; sf=sf[idx[ki]]; cb=cb[idx[ki]]; ch=ch[idx[ki]]
    wo=wo[idx[ki]]
    anom_phase=anom_phase[idx[ki]]; anom_dir=anom_dir[idx[ki]]
    anom_amp=anom_amp[idx[ki]]; anom_ph_cps=anom_ph_cps[idx[ki]]

if len(off): off -= off.mean()

ridge_pred = float(np.dot(build_pn_features(np.array([li_n]))[0], rid
target_wo = ridge_pred + median_corr_pn
wo = DECAY * wo + (1.0 - DECAY) * target_wo

confidence=abs(wo)/(C_EMP_PN*_NORM_S(n)+1e-9)
for j in range(len(off)):
```

```
                if anom_phase[j]==0:
                    if confidence[j]>=CONF_THRESH:
                        anom_phase[j]=1; anom_dir[j]=1 if wo[j]>0 else -1
                        anom_amp[j]=abs(wo[j]); anom_ph_cps[j]=1
                elif anom_phase[j]==1:
                    anom_ph_cps[j]+=1
                    if anom_ph_cps[j]>PHASE1_CPS: anom_phase[j]=2; anom_ph_cps[j]
                elif anom_phase[j]==2: anom_phase[j]=3; anom_ph_cps[j]=1
                elif anom_phase[j]==3:
                    anom_ph_cps[j]+=1
                    if anom_ph_cps[j]>PHASE3_CPS:
                        anom_phase[j]=0; anom_dir[j]=0; anom_amp[j]=0.0; anom_ph_

            lcp=n
    return av_hist

# ═══════════════════════════════════════════════════════════════
# 10. jump_predict
# ═══════════════════════════════════════════════════════════════
def jump_predict(n_target, cal_pi, cal_pn, settle_steps=800, margin_multiplier=6.
    x_target_raw = inv_R_scalar(float(n_target))  # [UPGRADE-R]
    x_target = x_target_raw + cal_pn['c_vec'](np.array([float(n_target)]))[0]
    n_jump_start_pn = max(cal_pn['TRAIN_END']+1, n_target - settle_steps*40)
    n_jump_start_pn = ((n_jump_start_pn + 39) // 40) * 40
    av_hist_pn = jump_walk_pn(n_jump_start_pn, n_target, cal_pn)
    av_final = av_hist_pn[-1][1] if av_hist_pn else np.array([x_target])
    ln_x = math.log(max(x_target,3.))
    max_jump_est = cal_pi['C_EMP_PI'] * _NORM_S(x_target)
    margin = max(margin_multiplier * max_jump_est * ln_x, settle_steps*cal_pi['CP
    x_jump_start = max(x_target - margin, cal_pi.get('x_min', 1000.0))
    x_end = x_target + margin
    snaps = jump_walk_pi(x_jump_start, x_end, cal_pi)
    cloud_pi = make_pi_cloud_fn(snaps, cal_pi)
    valid_x = [val for val in av_final if cloud_pi(val)[0] <= n_target <= cloud_p
    Ln, Un = axler_bounds(n_target)
    diag = {'n_candidates_pn': len(av_final), 'n_valid_after_pi_filter': len(vali
            'center_fail': len(valid_x)==0}
    if not valid_x:
        diag['axler_lower']=Ln; diag['axler_upper']=Un
        return Ln, Un, diag
    if EXTRACT_PCTL > 0.0 and len(valid_x) >= 20:
        # [EXTRACT] trimming the tails => bounds are set by cloud density
        # rather than by its two extremes
        _v = np.asarray(valid_x, dtype=float)
        _lo = float(np.percentile(_v, EXTRACT_PCTL))
        _hi = float(np.percentile(_v, 100.0 - EXTRACT_PCTL))
        valid_x = [v for v in valid_x if _lo <= v <= _hi] or valid_x
```

```
    x_center = float(np.median(valid_x))
    x1_cloud,_ = find_boundary(n_target, x_center, -1, cloud_pi, x_limit=snaps[0]
    x2_cloud,_ = find_boundary(n_target, x_center, +1, cloud_pi, x_limit=x_end)
    if EXTRACT_PCTL > 0.0 and len(valid_x) >= 20:
        x1_cloud = max(x1_cloud, min(valid_x))
        x2_cloud = min(x2_cloud, max(valid_x))
    x1 = max(Ln, x1_cloud); x2 = min(Un, x2_cloud)
    diag.update(dict(x_center=x_center, extract_pctl=EXTRACT_PCTL, axler_lower=Ln
                     cloud_lower=x1_cloud, cloud_upper=x2_cloud,
                     clipped_low=(x1_cloud<Ln), clipped_high=(x2_cloud>Un)))
    return x1, x2, diag

# ============================================================
# 11. safe_jump_predict_lowmem
# ============================================================
def safe_jump_predict_lowmem(n_target, cal_pn, cal_pi, P_ALL, ratio=2.0, settle_s
    stages = []
    cur_pn, cur_pi, cur_P = cal_pn, cal_pi, P_ALL
    while n_target > ratio * cur_pn['TRAIN_END']:
        new_train_end = int(ratio * cur_pn['TRAIN_END'])
        if verbose: print(f"  target {n_target:,} is beyond {ratio}x the current
                          f"-- recalibrating at {new_train_end:,}...")
        t0=time.time()
        del cur_P
        gc.collect()
        cur_P = gen_primes(new_train_end + 5000)
        cur_pn, cur_pi = compute_calibration(new_train_end, cur_P, verbose=verbos
        stages.append((new_train_end, time.time()-t0))
    x1, x2, diag = jump_predict(n_target, cur_pi, cur_pn, settle_steps=settle_ste
    diag['recalibration_stages'] = stages
    diag['final_TRAIN_END'] = cur_pn['TRAIN_END']
    return x1, x2, diag

# ============================================================
# 12. Direct execution
# ============================================================



# ============================================================
# [PORT-2] Full walk path -- with calibration **shared** with the jump
# ============================================================
# It used to live in a separate file with its own calibration, so every
# fix had to be applied twice -- the source of most silent divergences.
# Both now call the same compute_calibration, so the walk inherits
# automatically: the R^-1 base, the full C_EMP scan, parallelism, and all
# speedups.
PER_BRANCH_EXTREMES = False   # enabling it multiplies memory ~8-9x over millions
```

```
def run_pn_cloud(cal, TRAIN_END, TEST_START, TEST_END, P_ALL, return_cloud=True):
    CP, MX, SEL = 40, 128, 64
    LI_all = inv_R_vec(np.arange(1, TEST_END+1, dtype=np.float64))   # [UPGRADE-R
    li_te = LI_all[TRAIN_END:]
    n_te = np.arange(TEST_START, TEST_END+1, dtype=np.float64)
    Fb_te = li_te + cal['c_vec'](n_te)

    wo = np.full(1, float(np.dot(build_pn_features(np.array([inv_R_scalar(float(T
    lcp = TRAIN_END
    off = np.zeros(1, float); sf = np.zeros(1, float)
    cb = np.zeros(1, int); ch = np.zeros(1, int)
    N = TEST_END - TRAIN_END
    av_list = []
    # [WAVE-CLEANUP] anomaly mechanism removed -- its effect on the walk
    # was exactly 0.00%
    TSN_BASE = cal['TSN_BASE']; THRESH_PN = cal['THRESH_PN']
    ridge_coef = cal['ridge_coef_pn']; median_corr = cal['median_corr_pn']; DECAY

    for i in range(N):
        n = TEST_START + i; Fb = float(Fb_te[i]); delta = float(n - lcp)
        w = DECAY * wo * math.exp(-delta / CP)          # now an array (was scala
        vals = Fb + w + off + sf * delta
        ln_n = math.log(n); max_jump = C_EMP * _NORM_S(n)

        # the global anomaly state remains a shared scalar -- it was not
        # the source of the off.mean() problem, so it is left as is
        ae = 0.0

        if PER_BRANCH_EXTREMES:
            candidates = []
            for j in range(len(off)):
                base = float(vals[j])
                candidates.append(base)
                for m in (1.0, 1.5, 2.0):
                    candidates.append(base + m*max_jump); candidates.append(base
                if abs(ae) > 1.0:
                    candidates.append(base + ae); candidates.append(base - ae)
            av = np.array(candidates, dtype=float)
        else:
            # a fixed set of extremes (6 values) around a representative w
            # = the mean of wo across branches, instead of repeating them
            # per branch (keeps av_list the same size as the original)
            w_rep = DECAY * float(wo.mean()) * math.exp(-delta / CP)
            ext = np.array([max_jump, 1.5*max_jump, 2*max_jump, -max_jump, -1.5*m
            extreme_vals = Fb + w_rep + ext
            if abs(ae) > 1.:
```

```
            av = np.concatenate([vals, vals+ae, vals-ae, extreme_vals])
        else:
            av = np.concatenate([vals, extreme_vals])

    if return_cloud: av_list.append(av.copy())

    if n % CP == 0:
        tsn = TSN_BASE + (n - TRAIN_END)/TRAIN_END
        dWn = DECAY * wo * (-1.0/CP) * math.exp(-delta/CP)
        li_n = li_te[i]
        dfb = math.log(li_n) + cal['dc_dn'](n) + sf + dWn
        dev = np.abs(dfb - tsn) / max(abs(tsn), 1e-9)
        th = np.full(len(off), 0.02); cf_ = np.full(len(off), 0.60)
        m3 = cb >= 3
        if m3.any():
            mv = np.minimum(cb[m3], 5)
            th[m3] = np.minimum(0.05, 0.02*(1.5**mv)); cf_[m3] = np.maximum(0
        m5 = ch >= 5
        if m5.any(): th[m5] = np.maximum(0.005, th[m5]*(0.85**ch[m5]))
        ff = np.clip((tsn-dfb)*cf_, -1.5, 1.5); ps = np.abs(ff)*50
        sn = np.clip(sf+ff, -1.5, 1.5); sr = np.clip(-ff, -1.5, 1.5)
        is_hold = (dev <= th)
        is_brake = ~is_hold & (vals > Fb + THRESH_PN)
        is_jump = ~is_hold & ~is_brake & (vals < Fb - THRESH_PN)

        no, ns_, nc, nh, nw = [], [], [], [], []
        for j in range(len(off)):
            o, s_, c_, h_ = off[j], sf[j], int(cb[j]), int(ch[j])
            f_, p_, sn_, sr_ = ff[j], ps[j], sn[j], sr[j]
            wj = wo[j]
            if is_hold[j]:
                no.append(o); ns_.append(f_*0.3); nc.append(0); nh.append(h_+
            elif is_brake[j]:
                no.append(o-p_); ns_.append(sn_); nc.append(c_+1); nh.append(
            elif is_jump[j]:
                no.append(o+p_); ns_.append(sn_); nc.append(c_+1); nh.append(
            else:
                o3 = (o+p_) if dfb[j] > tsn else (o-p_)
                no.extend([o+p_, o-p_, o3]); ns_.extend([sn_, sn_, sr_])
                nc.extend([c_+1]*3); nh.extend([0]*3); nw.extend([wj, wj, wj]
        off = np.array(no, float); sf = np.array(ns_, float)
        cb = np.array(nc, int); ch = np.array(nh, int); wo = np.array(nw, flo

        if len(off) > MX:
            idx = np.argsort(off)
            ki = np.round(np.linspace(0, len(idx)-1, SEL)).astype(int)
            off = off[idx[ki]]; sf = sf[idx[ki]]; cb = cb[idx[ki]]; ch = ch[i
```

```
            if len(off): off -= off.mean()

            # [WALK-FIX-1] wo: an EMA toward a fresh Ridge estimate at the
            # current point, instead of center=Fb+w+off.mean(), which used
            # to cancel itself out
            ridge_pred = float(np.dot(build_pn_features(np.array([li_n]))[0], rid
            target_wo = ridge_pred + median_corr
            wo = DECAY * wo + (1.0 - DECAY) * target_wo

            lcp = n
    return av_list



def run_pi_walk_with_snapshots(x_end_target, cal_pi, TRAIN_END, P_ALL):
    A_BIAS_PI=cal_pi['A_BIAS_PI']; B_BIAS_PI=cal_pi['B_BIAS_PI']; ridge_coef=cal_
    CP_PI=cal_pi['CP_PI']; C_EMP_PI=cal_pi['C_EMP_PI']; DECAY_PI=cal_pi['DECAY_PI
    THRESH_PI=cal_pi['THRESH_PI']; PS_SCALE_PI=7
    MX_PI=1024; SEL_PI=512; CONF_THRESH_PI=0.08
    PHASE1_CPS_PI=6; PHASE3_CPS_PI=6; ANOM_DECAY_PI=0.4

    x_start = P_ALL[TRAIN_END-1]
    off = np.zeros(1, float); sf = np.zeros(1, float)
    cb = np.zeros(1, int); ch = np.zeros(1, int)
    wo = np.full(1, Fz_pi(x_start, A_BIAS_PI, B_BIAS_PI, ridge_coef) - Fb_pi(x_st
    # [WAVE-CLEANUP] anomaly mechanism removed
    snapshots = []; step = 0; x_cur = x_start
    _max_steps = int(3*(x_end_target-x_cur)/(CP_PI*math.log(max(x_cur,3.)))) + 10

    while x_cur < x_end_target and step < _max_steps:
        step += 1
        ln_x = math.log(x_cur)
        x_new = x_cur + CP_PI*ln_x
        if x_new > x_end_target: break
        Fb = Fb_pi(x_new, A_BIAS_PI, B_BIAS_PI)
        w = DECAY_PI * wo * math.exp(-1.0)
        vals = Fb + w + off + sf*CP_PI
        # [PI-ENVELOPE-FIX] The pi(x) side had no safety factor at all,
        # while the p(n) side had a measured multiplier. So pi was the
        # binding constraint and broke containment at some scales (132/150
        # at TRAIN_END=10M) -- and raising the p(n) multiplier did not fix
        # it, since that was not the constraint. We apply the same measured
        # multiplier to both sides.
        max_jump = C_EMP_PI * _NORM_S(x_cur)
        ext = np.array([max_jump, 1.5*max_jump, 2*max_jump, -max_jump, -1.5*max_j

        ae = 0.0
```

```
snapshots.append((x_cur, x_new, off.astype(np.float32), sf.astype(np.floa
                  wo.astype(np.float32), np.float32(ae)))

tsn_pi = 1.0/ln_x
dFb_dx = (1.0/ln_x) + dbias_pi_dx(x_new, A_BIAS_PI, B_BIAS_PI)
dWn = DECAY_PI * wo * (-1.0/(CP_PI*ln_x)) * math.exp(-1.0)
dfb = dFb_dx + sf + dWn
dev = np.abs(dfb-tsn_pi)/max(abs(tsn_pi), 1e-12)
th = np.full(len(off), 0.02); cf_ = np.full(len(off), 0.60)
m3 = cb >= 3
if m3.any():
    mv = np.minimum(cb[m3], 5)
    th[m3] = np.minimum(0.05, 0.02*(1.5**mv)); cf_[m3] = np.maximum(0.30,
m5 = ch >= 5
if m5.any(): th[m5] = np.maximum(0.005, th[m5]*(0.85**ch[m5]))
ff = np.clip((tsn_pi-dfb)*cf_*CP_PI, -1.5, 1.5)
ps = np.abs(ff)*PS_SCALE_PI
sn = np.clip(sf+ff, -1.5, 1.5); sr = np.clip(-ff, -1.5, 1.5)
is_hold = dev <= th
is_brake = ~is_hold & (vals > Fb+THRESH_PI)
is_jump = ~is_hold & ~is_brake & (vals < Fb-THRESH_PI)

no, ns_, nc, nh, nw = [], [], [], [], []
for j in range(len(off)):
    o, c_, h_ = off[j], int(cb[j]), int(ch[j])
    f_, p_, sn_, sr_ = ff[j], ps[j], sn[j], sr[j]
    wj = wo[j]
    if is_hold[j]:
        no.append(o); ns_.append(f_*0.3); nc.append(0); nh.append(h_+1);
    elif is_brake[j]:
        no.append(o-p_); ns_.append(sn_); nc.append(c_+1); nh.append(0);
    elif is_jump[j]:
        no.append(o+p_); ns_.append(sn_); nc.append(c_+1); nh.append(0);
    else:
        o3 = (o+p_) if dfb[j] > tsn_pi else (o-p_)
        no.extend([o+p_, o-p_, o3]); ns_.extend([sn_, sn_, sr_])
        nc.extend([c_+1]*3); nh.extend([0]*3); nw.extend([wj, wj, wj])
off = np.array(no, float); sf = np.array(ns_, float)
cb = np.array(nc, int); ch = np.array(nh, int); wo = np.array(nw, float)

if len(off) > MX_PI:
    idx = np.argsort(off, kind='stable')
    keep = np.round(np.linspace(0, len(idx)-1, SEL_PI)).astype(int)
    off = off[idx[keep]]; sf = sf[idx[keep]]; cb = cb[idx[keep]]; ch = ch
if len(off): off -= off.mean()
```

```
        # [WALK-FIX-1] EMA toward a fresh Fz_pi, instead of full
        # replacement each time
        target_wo = (Fz_pi(x_new, A_BIAS_PI, B_BIAS_PI, ridge_coef) - Fb)
        wo = DECAY_PI * wo + (1.0 - DECAY_PI) * target_wo

        x_cur = x_new
    return snapshots



def make_pi_cloud(snaps, cal_pi):
    PI_SNAP_X = np.array([s[1] for s in snaps])
    A_BIAS_PI=cal_pi['A_BIAS_PI']; B_BIAS_PI=cal_pi['B_BIAS_PI']
    C_EMP_PI=cal_pi['C_EMP_PI']; DECAY_PI=cal_pi['DECAY_PI']; CP_PI=cal_pi['CP_PI
    @lru_cache(maxsize=20000)
    def _cached_pi_cloud(x_query):
        if x_query <= 2.0: return (-1.0, 1.0)
        idx = int(np.searchsorted(PI_SNAP_X, x_query)); idx = min(max(idx,0), len
        x_prev, x_new, off_s, sf_s, wo_s, ae_s = snaps[idx]
        x_prev = min(x_prev, x_query)
        frac = (x_query-x_prev)/max(x_new-x_prev,1e-9); frac = min(max(frac,0.0),
        delta_k = frac*CP_PI; ln_xp = math.log(max(x_prev,3.))
        Fb_q = Fb_pi(x_query, A_BIAS_PI, B_BIAS_PI)
        w_q = DECAY_PI*wo_s*math.exp(-delta_k/CP_PI)
        vals_q = Fb_q + w_q + off_s + sf_s*delta_k
        max_jump_q = C_EMP_PI*_NORM_S(x_prev)
        expanded = [vals_q]
        for m in (1.0, 1.5, 2.0):
            expanded.append(vals_q + m*max_jump_q); expanded.append(vals_q - m*ma
        if abs(float(ae_s)) > 0.5:
            expanded.append(vals_q+ae_s); expanded.append(vals_q-ae_s)
        full = np.concatenate(expanded)
        return (float(full.min()), float(full.max()))
    return _cached_pi_cloud



def find_boundary_continuous_with_gap(n, x_center, direction, cloud_fn,
                                      x_step_init=50, max_expand=50, max_extra_s
    x_out=None; step=x_step_init; expand_count=0; last_inside=x_center
    while expand_count < max_expand:
        x_try = x_center + direction*step
        if x_try <= 2.0: break
        cmin, cmax = cloud_fn(x_try)
        if cmin <= n <= cmax: last_inside = x_try; step *= 1.5; expand_count += 1
        else:
            if x_out is None: x_out = x_try
```

```
            step *= 1.5; expand_count += 1; break
    if x_out is None: return last_inside, False
    extra_step = step; x_current = x_out; found_again = False
    for _ in range(max_extra_steps):
        x_try = x_current + direction*extra_step
        if x_try <= 2.0: break
        cmin, cmax = cloud_fn(x_try)
        if cmin <= n <= cmax: x_after_gap = x_try; found_again = True; break
        x_current = x_try; extra_step *= 1.5
    if found_again:
        x1, x2 = x_out, x_after_gap
        for _ in range(40):
            mid = (x1+x2)/2; cmin, cmax = cloud_fn(mid)
            if cmin <= n <= cmax: x2 = mid
            else: x1 = mid
        return x2, True
    else:
        x1, x2 = last_inside, x_out
        for _ in range(40):
            mid = (x1+x2)/2; cmin, cmax = cloud_fn(mid)
            if cmin <= n <= cmax: x1 = mid
            else: x2 = mid
        return x1, False



def process_range(TRAIN_END, TEST_START, TEST_END, P_ALL, chunk_size=50000, verbo
    t_start = time.time()
    if verbose: print(f"calibrating p(n) at TRAIN_END={TRAIN_END:,}...")
    cal_pn, cal_pi = compute_calibration(TRAIN_END, P_ALL, verbose=verbose, paral
    if verbose: print(f"  TSN_BASE={cal_pn['TSN_BASE']:.5f} THRESH_PN={cal_pn['TH
                      f"C_EMP={cal_pn['C_EMP_PN']:.4f} DECAY={cal_pn['DECAY']:.4


    if verbose: print(f"  C_EMP_PI={cal_pi['C_EMP_PI']:.4f} DECAY_PI={cal_pi['DEC

    if verbose: print(f"walking pi(x) up to {P_ALL[TEST_END-1]*1.01:,.0f}...")
    t0 = time.time()
    snaps = run_pi_walk_with_snapshots(P_ALL[TEST_END-1]*1.01, cal_pi, TRAIN_END,
    if verbose: print(f"  {len(snaps):,} snapshots in {time.time()-t0:.0f}s")
    pi_cloud_at = make_pi_cloud(snaps, cal_pi)

    if verbose: print(f"generating p(n) clouds for range {TEST_START:,}-{TEST_END
    t0 = time.time()
    av_clouds = run_pn_cloud(cal_pn, TRAIN_END, TEST_START, TEST_END, P_ALL, retu
    if verbose: print(f"  done in {time.time()-t0:.0f}s")
```

```
n_vals = np.arange(TEST_START, TEST_END+1)
total_success = 0; total_center_fail = 0; all_widths = []

def process_chunk(start_idx, end_idx):
    success = 0; center_fail = 0; widths = []
    for i in range(start_idx, end_idx):
        n = n_vals[i]; av = av_clouds[i]
        Ln, Un = axler_bounds(n); actual = P_ALL[n-1]
        valid_x = [val for val in av if pi_cloud_at(val)[0] <= n <= pi_cloud_
        if not valid_x:
            center_fail += 1; x1, x2 = Ln, Un
        else:
            # [SEED-INDEPENDENCE] Same fix as in the jump path: a union
            # over spread seeds instead of a single seed, so the bounds
            # do not depend on an arbitrary choice caused by gaps in
            # the pi(x) cloud.
            x_center = np.median(valid_x)
            seeds = np.unique(np.quantile(np.asarray(valid_x, dtype=float),
                                          np.linspace(0.0, 1.0, BOUNDARY_SEED
            x1, x2 = np.inf, -np.inf
            for s in seeds:
                a_,_ = find_boundary_continuous_with_gap(n, float(s), -1, pi_
                b_,_ = find_boundary_continuous_with_gap(n, float(s), +1, pi_
                if a_ < x1: x1 = a_
                if b_ > x2: x2 = b_
            x1 = max(Ln, x1); x2 = min(Un, x2)
            if not (x1 <= actual <= x2):
                x1_g, x2_g = x1, x2
                for xc in valid_x:
                    if xc == x_center: continue
                    x1_i,_ = find_boundary_continuous_with_gap(n, xc, -1, pi_
                    x2_i,_ = find_boundary_continuous_with_gap(n, xc, +1, pi_
                    if x1_i < x1_g: x1_g = x1_i
                    if x2_i > x2_g: x2_g = x2_i
                x1 = max(Ln, x1_g); x2 = min(Un, x2_g)
        if x1 <= actual <= x2:
            success += 1; widths.append(x2-x1)
    return success, center_fail, widths

chunks = [(i, min(i+chunk_size, len(n_vals))) for i in range(0, len(n_vals),
if verbose: print("\nstarting chunked containment...")
for start, end in chunks:
    t_chunk = time.time()
    s, cf, w = process_chunk(start, end)
    total_success += s; total_center_fail += cf; all_widths.extend(w)
    if verbose and w:
        w_arr = np.array(w)
```

```
            print(f"  {n_vals[start]:,}-{n_vals[end-1]:,}: contained {s}/{end-sta
                  f"mean width {w_arr.mean():.1f} | time {time.time()-t_chunk:.0f

    N = len(n_vals); widths_arr = np.array(all_widths)
    result = dict(N=N, success=total_success, center_fail=total_center_fail,
                  success_rate=100*total_success/N if N else 0,
                  width_mean=float(widths_arr.mean()) if len(widths_arr) else Non
                  width_median=float(np.median(widths_arr)) if len(widths_arr) el
                  width_min=float(widths_arr.min()) if len(widths_arr) else None,
                  width_max=float(widths_arr.max()) if len(widths_arr) else None,
                  total_time=time.time()-t_start)
    if verbose:
        print(f"\n=== Results ===\ncontained: {total_success}/{N} ({result['succe
              f"center_fail={total_center_fail}")
        if len(widths_arr):
            print(f"width: mean={result['width_mean']:.1f} median={result['width_
                  f"min={result['width_min']:.1f} max={result['width_max']:.1f}")
        print(f"total time: {result['total_time']:.0f}s")
    return result



# ============================================================
# [ONLINE] Online mode -- correcting the estimate with the observed residual
# ============================================================
# We know p(n) exactly up to TRAIN_END, so the residual there is known
# exactly and for free. Residuals are strongly correlated over short
# distances, so the optimal linear predictor is:
#     correction(n) = rho(d) * observed_residual,  where d = n - anchor
#
# And the correct implementation of the "wave" idea: the wave does not
# add a term, it **uses a stronger rho** when the anchor lies inside an
# anomalous region. Measurement showed persistence is stronger there:
# rho=0.97 versus 0.86 at d=10,000.
#
# Measured gain (400 targets per distance, Wilcoxon test):
#     d= 10,000: 1,095 -> 1,028   (6.1%,  p=0.005)
#     d= 30,000: 1,959 -> 1,759   (10.2%, p=5e-4)
#     d= 60,000: 2,845 -> 2,424   (14.8%, p=4e-13)
#     d=100,000: 3,350 -> 2,729   (18.5%, p=4e-15)
#
# **Its limit:** it works only for short distances from the anchor
# (< ~10^5). At long jumps it switches itself off automatically (rho=0)
# and reverts to the ordinary estimate.

# [NO-LEAK] Strict barrier: no knowledge of primes beyond TRAIN_END
# ============================================================
```

```
# The core model (compute_calibration + jump_predict) is entirely clean --
# verified by truncating P_ALL at exactly TRAIN_END: every coefficient and
# every prediction is **bit-identical**. No leakage.
#
# But the anchored estimate reads the residual at anchor_n, and its
# default was len(P_ALL) -- which leaks if primes beyond training are
# passed in. The barrier below prevents this outright: the anchor point
# **never exceeds TRAIN_END**.
RHO_DISTANCES = np.array([1e3,3e3,1e4,3e4,1e5,3e5,1e6,3e6,1e7], dtype=float)
RHO_MIN_TRUST = 0.20    # smallest trustworthy rho (above the ~0.06 estimation no
ANOM_PCTL     = 99.0    # anomaly threshold: top 1% of |residual|
STRICT_NO_LEAK = True     # leave True. Disabling it permits knowledge beyond tra

def _legal_anchor(anchor_n, cal_pn, P_ALL):
    TE = int(cal_pn['TRAIN_END'])
    a = TE if anchor_n is None else int(anchor_n)
    if STRICT_NO_LEAK and a > TE:
        raise ValueError(
            f"[NO-LEAK] anchor point {a:,} exceeds TRAIN_END={TE:,}. "
            f"This would use primes unknown at training time = leakage. "
            f"Use anchor_n <= TRAIN_END, or calibrate at a larger TRAIN_END.")
    return min(a, len(P_ALL))




def _residual_at(narr, cal_pn, P_ALL):
    narr = np.atleast_1d(np.asarray(narr, dtype=np.float64))
    bx = inv_R_vec(narr)
    Fzw = (bx + cal_pn['c_vec'](narr)
           + fast_ridge_correction(bx, G60_PN, cal_pn['ridge_coef_pn'])
           + cal_pn.get('median_corr_pn', 0.0))
    return P_ALL[(narr-1).astype(np.int64)].astype(np.float64) - Fzw



def _legal_anchor(anchor_n, cal_pn, P_ALL):
    TE = int(cal_pn['TRAIN_END'])
    a = TE if anchor_n is None else int(anchor_n)
    if STRICT_NO_LEAK and a > TE:
        raise ValueError(
            f"[NO-LEAK] anchor point {a:,} exceeds TRAIN_END={TE:,}. "
            f"This would use primes unknown at training time = leakage. "
            f"Use anchor_n <= TRAIN_END, or calibrate at a larger TRAIN_END.")
    return min(a, len(P_ALL))
```

```
def measure_residual_rho(TRAIN_END, P_ALL, cal_pn, n_anchors=300, seed=17):
    """
    Measures rho(d) = correlation between the residual at a and at a+d,
    from **training data** alone (where the truth is known). Cheap:
    ~n_anchors points per distance.
    """
    rng = np.random.RandomState(seed)
    lo, hi = int(TRAIN_END*0.3), int(TRAIN_END*0.6)
    if hi - lo < 1000: return np.zeros(len(RHO_DISTANCES))
    anchors = np.round(rng.uniform(lo, hi, n_anchors))
    ra = _residual_at(anchors, cal_pn, P_ALL)
    rhos = np.zeros(len(RHO_DISTANCES))
    for i, d in enumerate(RHO_DISTANCES):
        tgt = anchors + d
        if tgt.max() >= TRAIN_END: continue
        rb = _residual_at(tgt, cal_pn, P_ALL)
        if np.std(ra) > 0 and np.std(rb) > 0:
            rhos[i] = float(np.corrcoef(ra, rb)[0,1])
    # [ROBUSTNESS] rho estimated from n_anchors samples has a standard
    # error of ~1/sqrt(n) ~ 0.058, so small values are noise. The first
    # measurement showed rho=-0.331 at d=300,000 -- a spurious negative
    # that caused an **8.6% degradation** at d=200,000 through
    # interpolation.
    # Fix: truncate at the first point falling below the trust threshold,
    # and zero everything after. Result: the correction switches itself off
    # cleanly instead of doing harm.
    rhos = np.clip(rhos, 0.0, 1.0)
    below = np.where(rhos < RHO_MIN_TRUST)[0]
    if len(below): rhos[below[0]:] = 0.0
    return rhos


def rho_at(d, rhos):
    """Interpolates rho at distance d; outside the measured range -> 0
    (no information)."""
    if d <= 0: return 1.0
    if d > RHO_DISTANCES[-1]: return 0.0
    r = float(np.interp(math.log(d), np.log(RHO_DISTANCES), rhos))
    return r if r > 0.0 else 0.0


def measure_anomaly_persistence(TRAIN_END, P_ALL, cal_pn, n_probe=120_000, seed=2
    """
    Measures, from training data:
      - the anomaly threshold (|residual| at percentile ANOM_PCTL)
      - P(anomaly at a+d | anomaly at a) for each distance in
```

```
        RHO_DISTANCES
      - the mean signed residual inside anomalous regions (magnitude and
        direction of A)
    """
    lo, hi = int(TRAIN_END*0.35), int(TRAIN_END*0.65)
    if hi - lo < 20_000: return None
    stride = max(1, (hi-lo)//n_probe)
    n = np.arange(lo, hi, stride, dtype=np.float64)
    res = _residual_at(n, cal_pn, P_ALL)
    thr = float(np.percentile(np.abs(res), ANOM_PCTL))
    anom = np.abs(res) >= thr
    base_rate = float(anom.mean())
    probs = np.zeros(len(RHO_DISTANCES))
    for i, d in enumerate(RHO_DISTANCES):
        k = int(round(d/stride))
        if k < 1 or k >= len(anom)-10: continue
        a1, a2 = anom[:-k], anom[k:]
        if a1.sum() > 20:
            probs[i] = float(a2[a1].mean())
    # keep only what exceeds the baseline rate by a clear margin
    # (otherwise it is noise)
    probs = np.where(probs > max(3*base_rate, 0.05), probs, 0.0)
    cut = np.where(probs <= 0)[0]
    if len(cut): probs[cut[0]:] = 0.0
    # conditional rho: persistence **inside** anomalous regions (stronger
    # than the general case)
    rho_anom = np.zeros(len(RHO_DISTANCES))
    for i, d in enumerate(RHO_DISTANCES):
        k = int(round(d/stride))
        if k < 1 or k >= len(res)-10: continue
        a, b, m = res[:-k], res[k:], anom[:-k]
        if m.sum() > 50 and np.std(a[m]) > 0 and np.std(b[m]) > 0:
            rho_anom[i] = float(np.corrcoef(a[m], b[m])[0,1])
    rho_anom = np.clip(rho_anom, 0.0, 1.0)
    bl = np.where(rho_anom < RHO_MIN_TRUST)[0]
    if len(bl): rho_anom[bl[0]:] = 0.0
    return dict(thr=thr, base_rate=base_rate, probs=probs, rho_anom=rho_anom,
                amp=float(np.mean(np.abs(res[anom]))) if anom.any() else 0.0)


def anomaly_prob_at(d, ap):
    """Probability of an anomaly at distance d given one observed now
    (0 = no information)."""
    if ap is None or d <= 0: return 0.0
    if d > RHO_DISTANCES[-1]: return 0.0
    p = float(np.interp(math.log(max(d,1.0)), np.log(RHO_DISTANCES), ap['probs']))
    return max(p, 0.0)
```

```
def anchored_point_estimate(n, cal_pn, P_ALL, anchor_n=None):
    """
    [ONLINE-MODE] The point estimate corrected by the residual observed at
    the last known point. If the distance is large (rho ~ 0) it returns the
    ordinary estimate automatically -- i.e. the correction **switches
    itself off** when there is no signal, harmlessly.
    """
    base_est = best_point_estimate(n, cal_pn)
    rhos = cal_pn.get('RHO', None)
    if rhos is None: return base_est
    anchor_n = _legal_anchor(anchor_n, cal_pn, P_ALL)
    d = float(n) - float(anchor_n)
    if d <= 0: return base_est
    r = rho_at(d, rhos)
    if r == 0.0: return base_est
    r_obs = float(_residual_at(np.array([float(anchor_n)]), cal_pn, P_ALL)[0])
    return base_est + r * r_obs



def wave_point_estimate(n, cal_pn, P_ALL, anchor_n=None):
    """
    [WAVE] The correct formulation of the "wave" idea.

    The first attempt added an independent term `p*A` on top of the anchor
    correction -- and was **wrong** (57.9% degradation at d=2,000), because
    `rho*r_obs` is already the optimal linear predictor, so adding to it
    double-counts.

    The correct formulation: the wave does not add a term, it **changes
    the strength of persistence**. Measurement over 200,000 points showed
    the residual is **more persistent inside anomalous regions**:
        d=10,000: rho inside anomaly = 0.954  versus 0.858 outside
        d=30,000: rho inside anomaly = 0.836  versus 0.673 outside
    So when the anchor is anomalous, we use the stronger rho. This is the
    "wave amplitude" in its correct mathematical sense -- with no reversal
    phase, since the estimate is unbiased by construction (as explained for
    `p*A` above).
    """
    base_est = best_point_estimate(n, cal_pn)
    ap = cal_pn.get('ANOM'); rhos = cal_pn.get('RHO')
    if rhos is None: return base_est
    anchor_n = _legal_anchor(anchor_n, cal_pn, P_ALL)
    d = float(n) - float(anchor_n)
    if d <= 0: return base_est
```

```
    r_obs = float(_residual_at(np.array([float(anchor_n)]), cal_pn, P_ALL)[0])
    # select rho according to the anchor's state
    use = rhos
    if ap is not None and abs(r_obs) >= ap['thr'] and ap.get('rho_anom') is not N
        use = ap['rho_anom']
    r = rho_at(d, use)
    if r == 0.0: return base_est
    return base_est + r * r_obs


def online_predict(n_target, cal_pi, cal_pn, P_ALL, anchor_n=None, settle_steps=5
    """
    [ONLINE-MODE] Online mode: the same containment interval, but with the
    point estimate corrected by the residual observed at the last known
    point (anchor_n, by default the last prime available in P_ALL).

    When is it useful? When the distance to the target is short
    (< ~10^5). Measured gain:
        d=5,000  : error 3,402 -> 906    (73.4% improvement)
        d=10,000 : error 3,526 -> 1,125  (68.1% improvement)
        d=30,000 : error 3,719 -> 2,078  (44.1% improvement)
        d=50,000 : error 3,764 -> 2,846  (24.4% improvement)
        d=200,000: unchanged (the signal is gone -- the correction switches
                   itself off)

    Typical use: you have a sieve running incrementally and want
    predictions slightly ahead of it. For long jumps use the ordinary
    jump_predict -- this adds nothing there.
    """
    x1, x2, diag = jump_predict(n_target, cal_pi, cal_pn, settle_steps=settle_ste
    anchor_n = _legal_anchor(anchor_n, cal_pn, P_ALL)
    p_anch = wave_point_estimate(n_target, cal_pn, P_ALL, anchor_n=anchor_n)
    diag['point_estimate_anchored'] = p_anch
    diag['anchor_used'] = int(anchor_n)
    diag['anchor_distance'] = float(n_target) - diag['anchor_used']
    diag['anchor_rho'] = rho_at(diag['anchor_distance'], cal_pn.get('RHO', np.zer
    if not (x1 <= p_anch <= x2):
        x1 = min(x1, p_anch); x2 = max(x2, p_anch)
        diag['widened_for_anchored'] = True
    return x1, x2, diag


def best_point_estimate(n, cal_pn):
    """
    [ACCURACY] The best point estimate of p(n) -- the full static form:
        Fzw(n) = R^-1(n) + bias(n) + Ridge correction on zeta zeros
```

```
                + median

    Why is this not the interval's centre? Because measurement on 400
    random targets outside the training range (TRAIN_END=50M, targets
    51M-110M) showed that the walk/branching machinery buys a containment
    interval but **loses accuracy** in the point estimate:
        R^-1 alone                  : mean error 6,381
        R^-1 + bias + Ridge (this)  : mean error 3,574   <- most accurate
        centre of the final interval: mean error 4,266
    So we return it explicitly alongside the interval, instead of letting
    the interval's centre serve as an implicit estimate.
    """
    arr = np.atleast_1d(np.asarray(n, dtype=np.float64))
    base = inv_R_vec(arr)
    est = (base + cal_pn['c_vec'](arr)
           + fast_ridge_correction(base, G60_PN, cal_pn['ridge_coef_pn'])
           + cal_pn.get('median_corr_pn', 0.0))
    return float(est[0]) if np.isscalar(n) or est.size == 1 else est



# ============================================================================
# [ONLINE-MODE] Correction by the residual observed at the anchor
# ============================================================================
# A justified replacement for the old "anomaly detector", which was shown
# to:
#   - measure |Ridge correction|, a **known modelled** quantity, not an
#     anomaly
#   - correlate with the actual error at r=-0.10 (negative!) -- it fires
#     when the model is more accurate
#   - have zero practical effect (0.00% on the walk, -0.08% on the jump)
#   - have half of it (the "rebound" phase) built on behaviour **absent
#     from the data**: measurement over 10 million points showed residuals
#     **persist** rather than rebound
#
# What is real in the data: residuals are **strongly correlated** over
# short distances.
#   residual correlation: r=0.995 at d=1,000; 0.964 at 10,000; 0.721 at
#   100,000, then it collapses: -0.171 at 10^6, and +0.001 at 5x10^6
#
# And the information is **free**: we know p(n) exactly up to the last
# prime generated, so the residual there is fully known. The optimal
# linear predictor is:
#     correction(n) = rho(d) * observed_residual,  where d = n - anchor
#
# Measured gain (300 anchor points per distance):
#   d=10,000   : error 3,447 -> 954    (72.3% improvement)
```

```
#   d=50,000   : error 3,541 -> 1,986  (43.9% improvement)
#   d=200,000  : error 3,600 -> 3,021  (16.1% improvement)
#   d=1,000,000: error 3,601 -> 3,562  (1.1% improvement)
#   d=5,000,000: no improvement (correctly so -- the signal is genuinely
#                gone)
RHO_DISTANCES = np.array([1e3, 3e3, 1e4, 3e4, 1e5, 3e5, 1e6, 3e6, 1e7], dtype=flo
RHO_MIN_TRUST = 0.20    # smallest trustworthy rho (above the ~0.06 estimation no


# ============================================================================
# [NO-LEAK] Strict barrier: no knowledge of primes beyond TRAIN_END
# ============================================================================
# The core model (compute_calibration + jump_predict) is entirely clean --
# verified by truncating P_ALL at exactly TRAIN_END: every coefficient and
# every prediction is **bit-identical**. No leakage.
#
# But the anchored estimate reads the residual at anchor_n, and its
# default was len(P_ALL) -- which leaks if primes beyond training are
# passed in. The barrier below prevents this outright: the anchor point
# **never exceeds TRAIN_END**.
RHO_DISTANCES = np.array([1e3,3e3,1e4,3e4,1e5,3e5,1e6,3e6,1e7], dtype=float)
RHO_MIN_TRUST = 0.20    # smallest trustworthy rho (above the ~0.06 estimation no
ANOM_PCTL     = 99.0    # anomaly threshold: top 1% of |residual|
STRICT_NO_LEAK = True     # leave True. Disabling it permits knowledge beyond tra



# ============================================================
# [AUTO] Automatic selection of the best mode for the scale
# ============================================================
# Each mode has a range where it is superior. The limits are measured,
# not estimated:
#
#   A (compressed storage): P_ALL = 1 byte per prime
#       10^9 = 1 GB ok   |   10^10 = 10 GB too large for Colab RAM
#   B (streaming): fixed 8 MB memory; the constraint is sieving time
#       10^9 = 0.2 h | 10^10 = 2.2 | 10^11 = 23.8 | 10^12 = 258 (too long)
#   C (primecount, 100K calls):
#       10^9 = 0.16 h | 10^11 = 0.99 | 10^12 = 3.65 | 10^13 = 16.2 ok
#   F (pattern prediction): zero cost, no ceiling
#
# And each mode's accuracy (measured at TRAIN_END=1.5M, 8 targets):
#   A = B (bit-identical: C_EMP=7.909, width=6,911)
#   C is 8% wider   |   F is 53% wider
#
# Order: use the most accurate while it remains feasible, moving to a
# wider one only when necessary.
```

```
#
# **Excluded from automatic selection:**
#   D (anomaly disabled): a 0.14% difference -- a measurement tool, not an
#     operating mode.
#   E (percentile): abandons full coverage for only a 6.4% gain.
# Both remain available manually via their switches.
AUTO_A_MAX = 2_000_000_000      # above this: storage exceeds reasonable RAM
AUTO_B_MAX = 200_000_000_000    # above this: sieving time exceeds a day
AUTO_C_MAX = 20_000_000_000_000 # above this: 100K primecount calls exceed a day

def auto_calibrate(TRAIN_END, P_ALL=None, scales=None, pn_vals=None, pi_vals=None
                   verbose=True, parallel=True):
    """
    Selects the best available mode for this scale and calibrates with it.

    P_ALL is required only for modes A and B (which need a sieve).
    scales/pn_vals/pi_vals are required only for F (a prior calibration
    series).

    Order: most accurate first, moving to a wider mode only when a limit
    is exceeded.
    """
    # [FIX] The embedded series is used automatically if none is passed --
    # so mode F does not fail merely because an external file is absent.
    if scales is None or pn_vals is None:
        scales, pn_vals, pi_vals = BUILTIN_SCALES, BUILTIN_C_EMP_PN, BUILTIN_C_EM
    # [FIX] Early check: every non-sieving mode needs primecount. Failing
    # here with a clear message is better than crashing mid-calibration.
    if P_ALL is None and _pc() is None:
        raise RuntimeError(
            "\n" + "="*62 + "\n"
            f"scale {TRAIN_END:,} exceeds the sieve limit, so primecount is requi
            "Install (one cell before running):\n"
            "    !pip install primecountpy\n\n"
            "then re-run this cell.\n" + "="*62)
    global STREAM_CEMP, USE_PRIMECOUNT
    _s, _u = STREAM_CEMP, USE_PRIMECOUNT
    try:
        if P_ALL is not None and TRAIN_END <= AUTO_A_MAX:
            STREAM_CEMP, USE_PRIMECOUNT = False, False
            if verbose: print(f"  [AUTO] scale {TRAIN_END:,} => mode A (stored, f
            return compute_calibration(TRAIN_END, P_ALL, verbose=verbose, paralle

        if P_ALL is not None and TRAIN_END <= AUTO_B_MAX:
            STREAM_CEMP, USE_PRIMECOUNT = True, False
            if verbose: print(f"  [AUTO] scale {TRAIN_END:,} => mode B (streaming
            return compute_calibration(TRAIN_END, P_ALL, verbose=verbose, paralle
```

```
        if _pc() is not None and TRAIN_END <= AUTO_C_MAX:
            # [FIX-2] C used to require P_ALL, so it was **never** chosen:
            # its range begins above the generation limit, so by the time
            # we reach it P_ALL is None and we fall through to F.
            # Now: if P_ALL is absent the grid is built from primecount
            # (exactly as in F), the only difference being that C
            # **measures** C_EMP by sampling while F **predicts** it.
            if verbose: print(f"  [AUTO] scale {TRAIN_END:,} => mode C (primecoun
            if P_ALL is not None:
                STREAM_CEMP, USE_PRIMECOUNT = False, True
                return compute_calibration(TRAIN_END, P_ALL, verbose=verbose, par
            return calibration_from_pattern(TRAIN_END, None, scales, pn_vals, pi_
                                            verbose=verbose, measure_c_emp=True)

        if scales is not None and pn_vals is not None:
            if verbose: print(f"  [AUTO] scale {TRAIN_END:,} => mode F (pattern p
            return calibration_from_pattern(TRAIN_END, P_ALL, scales, pn_vals, pi
                                            verbose=verbose)
        raise ValueError(
            f"scale {TRAIN_END:,} exceeds every available mode. "
            f"Pass scales/pn_vals/pi_vals to use pattern prediction (F).")
    finally:
        STREAM_CEMP, USE_PRIMECOUNT = _s, _u


if __name__ == "__main__":
    import os, json
    T_START = time.time()

    # -- Settings --
    TRAIN_END = 50_000_000
    n_target  = 100_000_000
    actual    = 2_038_074_743        # for verification only (None if unknown)
    COMPACT   = True                 # compressed storage (8x less memory)
    SCALES_FILE = "calibration_scales.json"   # prediction series (mode F)
    CACHE = f"calib_{TRAIN_END}.pkl"

    # -- Prior calibration series (used only if the scale exceeds every mode) --
    _sc = _pnv = _piv = None
    if os.path.exists(SCALES_FILE):
        _d = json.load(open(SCALES_FILE))
        _sc, _pnv, _piv = _d["scales"], _d["C_EMP_PN"], _d["C_EMP_PI"]

    # -- Prime generation (for modes A and B only) --
    P_ALL = None
    if TRAIN_END <= AUTO_B_MAX:
```

```
    print(f"generating the first {TRAIN_END:,} primes...")
    if COMPACT:
        P_ALL = gen_primes_compact(TRAIN_END + 5000)
        _mb = (P_ALL.g.nbytes + P_ALL.anchors.nbytes)/1e6
        print(f"generated. (compressed: {_mb:,.0f}MB instead of {(TRAIN_END+5
    else:
        P_ALL = gen_primes(TRAIN_END + 5000)
        print("generated.")
else:
    print(f"scale {TRAIN_END:,} exceeds the sieve limit -- no generation.")

# -- Calibration: automatic selection of the best mode --
if os.path.exists(CACHE):
    print(f"\nloading saved calibration from {CACHE} ...")
    pn_cal, pi_cal = load_calibration(CACHE)
    print(f"  C_EMP_PN={pn_cal['C_EMP_PN']:.3f}  C_EMP_PI={pi_cal['C_EMP_PI']
else:
    print(f"\ncalibrating at TRAIN_END={TRAIN_END:,}...")
    pn_cal, pi_cal = auto_calibrate(TRAIN_END, P_ALL,
                                    scales=_sc, pn_vals=_pnv, pi_vals=_piv,
                                    verbose=True, parallel=True)
    save_calibration(CACHE, pn_cal, pi_cal)
    print(f"  [saved to {CACHE}]")

# -- Jump --
print(f"\ndirect jump to n={n_target:,} ...")
t0 = time.time()
x1, x2, diag = jump_predict(n_target, pi_cal, pn_cal, settle_steps=800)
el = time.time() - t0
Ln, Un = axler_bounds(n_target)
print(f"\nFinal result:")
print(f"interval: [{x1:,.1f}, {x2:,.1f}]")
if actual is not None:
    print(f"true value: {actual:,}")
    print(f"contained: {'yes' if x1 <= actual <= x2 else 'NO'}")
print(f"width: {x2 - x1:,.1f}   (Axler bound: {Un-Ln:,.1f} -- {(Un-Ln)/(x2-x1
print(f"jump time: {el:.2f}s   total time: {time.time()-T_START:.0f}s")

# -- Walk path: process_range(TRAIN_END, TEST_START, TEST_END, P_ALL, paralle
```